NTNU
Norwegian University of Science and Technology (NTNU)
Faculty of Information Technology and Electrical Engineering
Department of Engineering Cybernetics

Specialization Project

Candidate:	**Bukueva Elena**
Course:	**TTK4551 Specialization Project**
Title (English):	**Embedded AI for Real-Time Object Detection**
Title (Norwegian):	**Embedded KI for sanntids objektdeteksjon**

Thesis description: Embedded AI refers to integrating artificial intelligence directly into hardware devices so they can process data and make decisions locally, without relying on the cloud. This enables real-time performance, reduced power consumption, and improved privacy in applications such as IoT, robotics, and smart sensors. In this project, we aim to compare the performance of different different hardware configurations, and to evaluate how various software frameworks and optimization strategies influence the overall results.

The tasks will be:

1. Give a short introduction to Embedded AI and explain the different development steps for creating an embedded system
2. Based on a specific case, explore the performance using different embedded hardware and optimization strategies.

Start date:	11. August, 2025
Due date:	18. December, 2025
Thesis performed at:	Department of Engineering Cybernetics
Supervisor:	Professor Geir Mathisen, Dept. of Eng. Cybernetics

Abstract

This project investigates practical Edge-AI deployment for the Real-Time fastener detection on Raspberry Pi 4/5, with and without Hailo-8 AI accelerator. It targets a detection on a conveyer belt with motion $\approx 2cm/s$, using an IDS uEye camera with maximum 21 FPS capability. Lightweight YOLO variants (v8/v10/v11 -nano and -small) and small RT-DETR-R18 transformer model are trained via transfer learning on a domain-specific fastener dataset. Four classes of objects are used during the training: machine screws, wood screws, nuts and washers. An additional fifth class is used to identify incomplete parts of an object, when neither a human, not a model can correctly classify the object. The dataset is prepared with the help of Roboflow framework, and it consists of 2131 augmented 960x960 grayscale images. Models are exported to ONNX format and compiled with the Hailo toolchain (ONNX $\rightarrow$ HAR $\rightarrow$ HEF) using INT8 post-training quantization (PTQ) on a calibration set.
A discussion of the annotation format choices, compilation workflow and on-device inference constraints is provided in the literature overview section.
Evaluation performed based on standard detection metrics (IoU, AP, mAP@0.50 and mAPP@[0.5:0.95]) and runtime (latency/FPS). On GPU, all tested YOLO versions archived competitive accuracy on the domain-specific fastener dataset. While on Raspberry PI 5 CPU, real-time throughput is not met, Hailo accelerator shows significantly higher raw inference capacity, having hardware benchmarks $\approx 142FPS$, while Raspberry Pi 5 pipeline delivers $\approx 1FPS$. However, archieved inference accuracy after the performed quantization is visibly lower than the pure YOLO PyTorch checkpoint inference. The studies on image size and Non-Maximum Suppression (NMS) settings, and briefly analyzed quantization effects on small, fine-grained targets are reported.

Sammendrag

Dette prosjektet undersøker praktisk Edge-AI-utrulling for sanntidsdeteksjon av festekomponenter på Raspberry Pi 5, med og uten Hailo-8 AI-akselerator. Målet er deteksjon på et transportbånd med bevegelse $\approx 2cm/s$, ved bruk av et IDS uEye-kamera med maksimal kapasitet på 21 FPS. Lette YOLO-varianter (v8/v10/v11 -nano and -small) og en liten RT-DETR-R18-transformermodell trenes med overføringslæring på et domenespesifikt datasett for festekomponenter. Fire objektklasser brukes under treningen: maskinskruer, treskruer, muttere og skiver. En femte klasse brukes til å identifisere ufullstendige deler av objektet, der verken et menneske eller modellen kan klassifisere objektet korrekt. Datasettet er forberedt ved hjelp av Roboflow-rammeverket og består av 2131 augmenterte 960x960 gråtonebilder. Modellene eksporteres til ONNX-format og kompileres med Hailo-verktøykjeden (ONNX $\rightarrow$ HAR $\rightarrow$ HEF) ved bruk av INT8 kvantisering etter trening fra et kalibreringssett.
En diskusjon av valg av annotasjonsformat, kompilasjonsarbeidsflyt og begrensninger for inferens på enheten presenteres i litteraturoversiktskapittelet.
Evalueringen er gjennomført basert på standard deteksjonsmetrikker (IoU, AP, mAP@0.50 og mAP@[0.5:0.95]) og kjøretid (latens/FPS). På GPU oppnådde alle testede YOLO-versjoner konkurransedyktig nøyaktighet på det innsamlede datasettet. På Raspberry Pi 5 CPU oppnås imidlertid ikke sanntidsgjennomstrømning, mens Hailo-akseleratoren viser betydelig høyere rå inferenskapasitet, med maskinvare-benchmarks på $\approx 142FPS$, mens Raspberry Pi 5-rørledningen leverer $\approx 1FPS$. Den oppnådde inferensnøyaktigheten etter den utførte kvantiseringen er likevel merkbart lavere enn for ren YOLO-inferens fra PyTorch-sjekkpunkter. Det rapporteres også studier av bildestørrelse og NMS-innstillinger, samt en kort analyse av kvantiseringseffekter på små, finstrukturerte mål.

Contents

Abbreviations

AI	Artificial Intelligence
AABB	Axis-Aligned Bounding Box
AP	Average Precision
API	Application Programming Interface
ARM	Arm architecture (CPU family)
CCW	Counterclockwise
COCO	Common Objects in Context (dataset)
CPU	Central Processing Unit
DL	Deep Learning
ES (in OpenGL ES)	OpenGL for Embedded Systems
FN	False Negative
FP	False Positive
FP32	32-bit floating point
FLOPs	Floating-Point Operations
FPS	Frames Per Second
GFLOPs	Giga Floating-Point Operations
GPU	Graphics Processing Unit
HAR	Hailo Archive (intermediate model package)
HEF	Hailo Executable Format (compiled binary for Hailo device)
HN	Hailo Network (graph JSON inside HAR)
IDS	Imaging Development Systems (uEye camera vendor)
INT8	8-bit integer
IoU	Intersection over Union
LLM	Large Language Model
L2/L3	Level-2 / Level-3 CPU cache
mAP	mean Average Precision
NMS	Non-Maximum Suppression
NPZ	NumPy Zip archive (weights/tensors)
OBB	Oriented (Rotated) Bounding Box
ONNX	Open Neural Network Exchange
OpenGL ES	OpenGL for Embedded Systems
PTQ	Post-Training Quantization

RAM	Random Access Memory
SOTA	State-of-the-art
SDK	Software Development Kit
SoC	System on Chip
TOPS	Tera Operations Per Second
TP	True Positive
uEye	IDS camera/product line
VStream	(Hailo) Virtual Stream (I/O stream abstraction)

1 Introduction

1.1 Motivation

With the increase of interest in AI usage on small devices a problem of the on-deploy arises. In the past decade Artificial Intelligence field made a big progress in solving complex problems of Natural Language processing, Computer Vision and Robotics. However, most of the time such models require a lot of both, data and computing resources. For instance, the full training of state-of-the-art open source Large Language Model (LLM) - LLaMA (3 400 B) [1] would take approximately 125 million years on a Raspberry Pi (ARM Cortex A53) and a thousand GPU datasenter is required to train the model in a proper amount of working days. In many real production settings, such highly complex models are unnecessary. Instead, there is a strong demand for fast processing on small, resource-constrained devices. The trade-off between accuracy and speed becomes a central point of interest.

This work investigates the performance of compact object detection models for edge deployment, comparing CPU-only inference on a small device, standart computer and accelerator and exploring how software frameworks and optimization strategies affect results. As the primary deployment target, we use a Raspberry Pi 5, chosen for its balance of cost, good community support (camera drivers, Python packages, and tooling), official support for the Raspberry Pi AI Kit / AI HAT+ based on the Hailo-8 accelerator, and its popularity in scientific research. Some runtime results are also performed on a host CPU (AMD Ryzen 9 5900X 12-Core Processor). The project initially started on a Raspberry Pi 4 Model B, but it was soon replaced to Raspberry Pi 5 because Raspberry Pi 4 specific setup could not host the Hailo-8–based AI Kit and, in addition, it provides less RAM and lower CPU performance than the Raspberry Pi 5.

In this project, we focus on detecting fasteners on a conveyor belt. This setup is close to real industrial use cases and can be easily extended to other objects. It also does not require a very large or complex dataset, which allows us to concentrate on the hardware and deployment aspects instead of spending most of the effort on defining and engineering the vision task itself.

We use four types of objects for training: wood screws, metal screws, nuts, and washers. They differ in size and appear in various positions and orientations on the belt, which keeps the detection problem challenging. Object detection in general has a wide range of methods, from classical morphology and segmentation to modern transformer-

based architectures and 3D detectors. Detection problems are important in many real-world applications, so there is already a lot of prior work and many approaches that run on standard hardware. This gives us a solid foundation to extend the work to edge devices and to compare models of different sizes and deployment strategies.

We implement near real-time object detection on the Raspberry Pi 5 and cover the full processing pipeline: from data collection with a physical camera, through model training and quantization, to deployment on the Hailo-8 accelerator. We provide empirical measurements of the trade-offs between different detector architectures (YOLO-family models and RT-DETR), input resolutions, and quantization strategies. The proposed setup evaluates small detection models on a fastener dataset and characterizes their behavior under realistic time and memory constraints.

1.2 Limitations

The project is subject to several practical and methodological limitations:

- **Narrow application domain:** The detector is trained and evaluated only on four fastener classes: machine and wood screws, nuts and washers, in a single conveyer-belt setup. We did not perform classification within classes e.g. classification by an object type or classification in different from the conveyor belt environments.
- **Restricted hardware platform:** All on-device experiments are performed on a Raspberry Pi 5 with 8 GB RAM and a single Hailo-8 accelerator. No comparison is made to other edge accelerators or microcontrollers. The conclusions are specific to this hardware combinations.
- **Model Choice** The experiment comparison is restricted to YOLOv8, YOLOv10, YOLOv11 nano and small variants as well as RT-DETR-R18. Moreover, only classic model optmization strategies provided by Hailo compiler are explored.
- **Dataset constraints** The Transfer Learning is performed only on a limited to 2131 grayscale images 960×960 resolution. Although data augmentation is applied, the number of unique physical parts, lighting conditions remains limited compared to large public datasets.

1.3 Contributions

Within these limitations, the main contributions of this specialization project are:

- **Domain-specific dataset:** Collection and annotation of a grayscale fastener dataset captured with an IDS uEye industrial camera, including four main fastener classes and a fifth miscellaneous class for incomplete or ambiguous parts.
- **Model training and comparison:** Training and evaluation of seven detectors (YOLOv8n/s, YOLOv10n/s, YOLOv11n/s, and RT-DETR-R18) on the fastener dataset, including experiments with two input resolutions (640×640 and 960×960) and analysis of their precision, recall, and mAP scores.
- **Edge deployment pipeline:** Implementation of a deployment workflow from PyTorch checkpoints to ONNX, HAR, and HEF, followed by on-device inference on Raspberry Pi 5 and the Hailo-8 accelerator. This includes configuring post-training INT8 quantization and integrating Hailo's NMS post-processing.
- **Runtime benchmarking:** Systematic measurement of latency and throughput on the target hardware, comparing CPU-only inference on Raspberry Pi 5 with accelerated inference on Hailo-8, and relating these results to the throughput requirements of the conveyor-belt setup.
- **Practical guidelines:** A set of concrete recommendations and configuration examples for future Edge-AI projects at NTNU, covering camera setup, dataset preparation, model selection, and Hailo/ONNX toolchain usage.

1.4 Thesis Structure

The remainder of this report is organised as follows:

- **Chapter 2** presents the literature study, covering introduction to relevant datasets, annotation strategies, and object-detection architectures, with a focus on the YOLO family and RT-DETR.
- **Chapter 3** introduces the theoretical background, in particular the evaluation metrics used for object detection such as IoU, precision/recall, AP, and mAP.
- **Chapter 4** outlines the overall methodology and work plan, from model selection through dataset creation, training, quantization, and deployment.
- **Chapter 5** details the proposed solution and design, including the concrete dataset and augmentation pipeline, camera configuration, Raspberry Pi and Hailo-8 setup, and the compilation flow.
- **Chapter 6** presents the experimental setup and reports the main quantitative results for accuracy and runtime on the different hardware platforms.

- **Chapter 7** discusses the results, highlighting key trade-offs, failure modes, and the impact of model choice, resolution, and quantization on performance.
- **Chapter 8** concludes the thesis by summarising the main findings and reflecting on the extent to which the original goals have been met.
- **Chapter 9** outlines directions for future work, including model improvements, and deeper integration into a complete industrial sorting system - with a robotic-arm and compilation strategies research.
- **Chapter 10** provides a description of use of artificial intelligence
- **Appendix A** presents additional visual material from the physical conveyor-belt setup, including the fastener classes, the belt mechanism, and the Raspberry Pi 5 configuration used for demonstrations.
- **Appendix B** documents the full Hailo-8 compilation workflow, including ONNX export, parsing, quantization, calibration, and HEF generation, as well as the exact toolchain commands used in the deployment pipeline.

2 Literature Study

2.1 Introduction to Object Detection with Deep Learning Models

Artificial Intelligence (AI) is a broad term covering a set of algorithms and operations on matrices and vectors that enable computers to perform a big variety of tasks in language understanding and translation, object recognition, robotic performance etc. We will consider only models performing object detection tasks, mostly having CNN [2] and Transformer [3] architectures.

At a high level, a neural network model can be seen as a parameterized function

$$f_\theta : \mathbb{R}^{H \times W \times C} \to \mathcal{Y},$$

where input is an image with height H, width W, and C channels, and the output space $\mathcal{Y}$ depends on the task. For object detection, $\mathcal{Y}$ consists of a set of bounding boxes (see description in section 2.2) and class probabilities for each detected object.

The vector θ contains all trainable (optimized) parameters (weights and biases) of the network.

It is common to use terms "training" and "inference" [4]. By training we understand some sort of iteration through a set of examples, so the model "learns" coefficients, approximating the distribution of the dataset. The more examples we have, the bigger share of general distribution we cover. We start training with some state of parameters (weights) θ, which are mostly randomly initialized. During training, parameters are iteratively updated so model's outputs match the desired targets on a labeled dataset. This can be described as an optimization task, we iteratively search for coefficients, which minimise the difference of the chosen metric, computed on a model-labeled and human-labeled pictures. For example, for object detection we want to decrease a localization loss (how well the predicted boxes align with the true boxes) and a classification loss (how well the predicted classes match the true classes). Metrics used to evaluate the quality of the training are described in detail in the Chapter 3. During inference (forward pass), parameters are fixed, and model simply computes $f_\theta(x)$ for an input image x.

The size of a neural network can be characterized by the number of parameters and the amount of computation (e.g. FLOPs) required per forward pass. Larger models typically have higher capacity and can represent more complex functions, which often leads to better accuracy when sufficient training data is available.

However, on embedded devices such as Raspberry Pi 4/5, model size has several practical consequences:

- **Memory usage:** Large models may not fit into the limited RAM available on edge devices.
- **Latency:** More parameters and layers lead to higher inference time, reducing FPS and making real-time processing harder to achieve.
- **Energy consumption:** Heavier models require more operations and therefore more power, which is critical for battery-powered systems.

For Edge-AI applications, it is necessary to balance accuracy and model size/computational cost. This trade-off leads to the use of lightweight architectures (e.g. nano/small variants) and techniques such as quantization and hardware acceleration.

Training a complex detector requires large, diverse datasets and big compute resources. So, it is common to rely on transfer learning. In transfer learning, a model is first trained on a large dataset (such as COCO [5]) and then adapted to a specific task or domain by fine-tuning on a smaller, domain-specific dataset.

In the context of this project, transfer learning enables the use of YOLO and RT-DETR models pretrained on large datasets, which are then fine-tuned on the domain-specific fastener dataset. This approach reduces training time and improves accuracy compared to training an equivalent detector from random weights initialization, while still allowing the model to adapt to the specific visual properties of screws, nuts, washers, and miscellaneous parts.

On the the other ways to speed up the inference is the use of additional accelerators (NPU). A CPU (Central Processing Unit) is designed to handle a wide variety of tasks, with complex control logic, deep caches, and relatively few powerful cores. It is very flexible, but not so efficient at the dense matrix multiplications and convolutions that dominate modern deep neural networks. An NPU (Neural Processing Unit) is specialized

hardware built specifically for these operations. It typically uses many small compute units, highly parallel data paths, and low-precision arithmetic (for example INT8 instead of FP32) to achieve much higher throughput per watt for inference. Despite high speed for matrix multiplications, NPU is not as efficient as CPU for a big range of tasks and cannot be used as a full substitution. That is why, in this work, accelerator are considered only as an addition to a edge-devices.

2.2 Datasets and Annonation strategies for Detection Tasks

Object detection models localize objects on an input picture or on a set of video frames. Localization can be performed by different approaches, the most relative to this work are:

- Axis-aligned bounding box (AABB): (x, y, w, h), which is cheap to annotate and fast to predict.
- Oriented/rotated bounding box (OBB): apart from AABB an angle θ of the box is introduced, which is a bit costlier to label.
- Polygon: a list of vertices tracing the object boundary. Provides precise geometry, however is time-consuming to label.
- Pixel mask ("painted area") - most precise 2D shape, ideal for measurement and robotics, heaviest to annotate and train on.

Choosing the right localization strategy should be guided by the structure of the available pretrained weights. Training large detectors from scratch is both resource- and time-intensive. For example, Ultralytics estimates that training YOLOv5 on COCO (118k images, 300 epochs) on a single NVIDIA V100-16GB takes approximately $n/s/m/l/x \approx 1/2/4/6/8$ days, respectively [6]. Collecting and annotating a dataset of that scale is even more time-consuming. Consequently, it is preferable to start from weights pretrained on a dataset whose *labeling granularity* matches the task (e.g., axis-aligned boxes vs. oriented boxes vs. masks). If the required annotation is more detailed than the pretraining labels, an additional annotation effort is needed for transfer learning. Then, if axis-aligned bounding boxes (AABB) are too coarse for small or complex objects, a richer localization (e.g., oriented boxes or instance masks) to meet accuracy requirements would be a better choice.

Mentioned earlier COCO dataset [5] is the common detection benchmark. It is big enough, varied, well-annotated and permissively licensed. The dataset contains photos of 91 objects types with a total of 2.5 million labeled instances in 328k images.

COCO's [5] official detection annotations are axis-aligned boxes only (bbox: [x, y, width, height]). There are no oriented boxes in COCO, so the standard YOLOv8/YOLO11 [7] *detect* checkpoints (trained on COCO 2017) learn AABB by default:

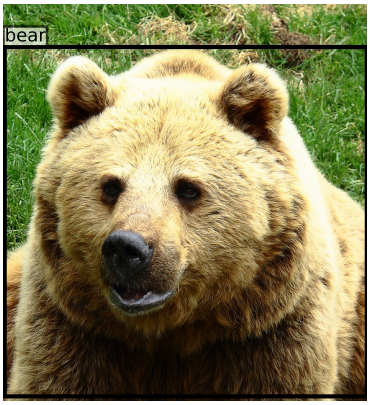


Figure 2.1: Example picture from the COCO dataset with AABB annotation [8]

Listing 2.1: COCO-style annotation entry

```
{
  "id": 1768,
  "image_id": 397133,
  "category_id": 1,
  "bbox": [73.0, 43.0, 199.0, 305.0],
  "area": 39220.0,
  "iscrowd": 0,
  "segmentation": [
    [78.0, 45.0, 86.0, 45.0, 95.0, 47.0, 101.0, 51.0, ...]
  ]
}
```

For oriented bounding boxes (OBB), the DOTA dataset [9] is the pioneering benchmark. Ultralytics provides YOLO `*-obb` weights pretrained on DOTA [10].

Other common datasets are:

- Objects365 [11] (600k images, 365 classes): broader coverage than COCO; popular for pretraining backbones/heads before COCO fine-tuning.
- Open Images [12] (millions of images; boxes, masks, relationships): huge scale, but label noise is higher; great for scale, weaker for cleanliness.

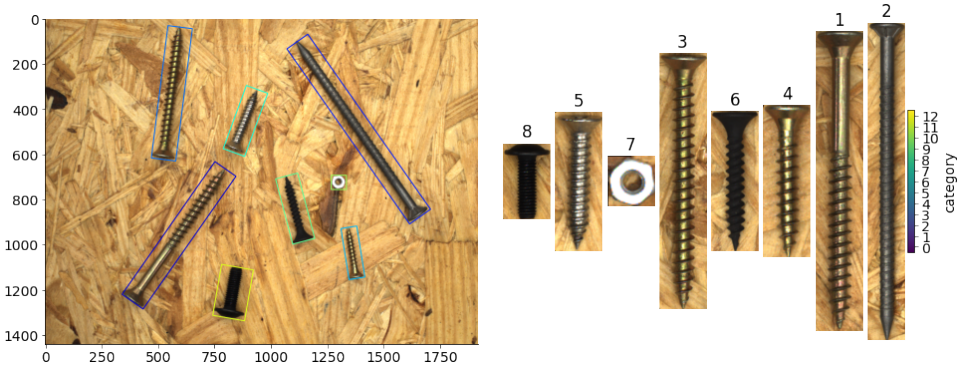


Figure 2.2: Example picture from the MVTec Screws dataset with OBB annotation [14]

None of these datasets, however, include "fasteners" as a class. In order to particularly train model on fasteners, a MVTec screws dataset [13] might be used. This dataset has oriented-box annotations for screws/nuts (384 images, 4,426 OBBs).

There are 13 categories used in the MVTec Screws oriented-box dataset (category_id → name) :

Table 2.1: MVTec Screws oriented-box categories

ID	Category
1	long lag screw
2	lag wood screw
3	wood screw
4	short wood screw
5	shiny screw
6	black oxide screw
7	nut
8	bolt
9	large nut
10	nut
11	nut
12	machine screw
13	short machine screw

It is also possible to convert OBB annotation to AABB annotation with some loss of information. In order to do so these steps can be followed:

Let w, h be side lengths and θ the rotation (radians, CCW). The tight axis-aligned

box around the rotated rectangle has

$$W_{\mathrm{AABB}} = |\cos\theta|\, w + |\sin\theta|\, h, \tag{2.1}$$

$$H_{\mathrm{AABB}} = |\sin\theta|\, w + |\cos\theta|\, h. \tag{2.2}$$

With the same center (c_x, c_y),

$$x_{\min} = c_x - \frac{W_{\mathrm{AABB}}}{2}, \qquad x_{\max} = c_x + \frac{W_{\mathrm{AABB}}}{2}, \tag{2.3}$$

$$y_{\min} = c_y - \frac{H_{\mathrm{AABB}}}{2}, \qquad y_{\max} = c_y + \frac{H_{\mathrm{AABB}}}{2}. \tag{2.4}$$

If the oriented box is given by four corners (x_i, y_i), the enclosing AABB is

$$x_{\min} = \min_i x_i, \quad x_{\max} = \max_i x_i, \qquad y_{\min} = \min_i y_i, \quad y_{\max} = \max_i y_i. \tag{2.5}$$

COCO bbox uses [$x_{\min}$, $y_{\min}$, width, height], then:

$$\mathrm{bbox}_{\mathrm{COCO}} = \big[\, x_{\min},\ y_{\min},\ x_{\max} - x_{\min},\ y_{\max} - y_{\min} \,\big]. \tag{2.6}$$

YOLO txt uses normalized $(x_{\mathrm{center}}, y_{\mathrm{center}}, w, h)$, then:
Given image size $(W_{\mathrm{img}}, H_{\mathrm{img}})$,

$$x_{\mathrm{center}} = \frac{x_{\min} + x_{\max}}{2\, W_{\mathrm{img}}}, \qquad y_{\mathrm{center}} = \frac{y_{\min} + y_{\max}}{2\, H_{\mathrm{img}}}, \tag{2.7}$$

$$w = \frac{x_{\max} - x_{\min}}{W_{\mathrm{img}}}, \qquad h = \frac{y_{\max} - y_{\min}}{H_{\mathrm{img}}}. \tag{2.8}$$

Another important step of the dataset formation is data augmentation. It is used to artificially increase the diversity of the training images without collecting new data. By applying simple transformations such as flips, crops, zooms, or changes in brightness and contrast, the model sees many different versions of the same objects. This helps it become more robust to variations in viewpoint, scale, lighting, and background that will appear in real deployments. In practice, augmentation reduces overfitting, improves generalization, and is especially important when the original dataset is relatively small or collected in a very controlled environment, as in our conveyor-belt setup.

Method	Train	mAP	FPS
Real-Time			
100Hz DPM [15]	2007	16.0	100
30Hz DPM [15]	2007	26.1	30
Fast YOLO	2007+2012	52.7	**155**
YOLO	2007+2012	**63.4**	45
Less Than Real-Time			
Fastest DPM [16]	2007	30.4	15
R-CNN Minus R [17]	2007	53.5	6
Fast R-CNN [18]	2007+2012	70.0	0.5
Faster R-CNN (VGG-16) [19]	2007+2012	73.2	7
Faster R-CNN (ZF) [19]	2007+2012	62.1	18
YOLO VGG-16	2007+2012	66.4	21

Table 2.2: Real-Time Systems on PASCAL VOC 2007. Fast YOLO is the fastest detector and YOLO attains higher mAP while remaining real-time. [7]

2.3 The model choice

For Edge AI, the trade-off between computation speed and memory availability is critical. A model must not only deliver sufficiently high accuracy, but also remain compact to fit onto resource-constrained devices such as microcontrollers.

YOLO models family (see Table 2.4) presents state-of-the-art lightweight versions of model small and efficient enough to be used on Edge-devices. The family shows good results for both - prediction accuracy and inference speed.

Before YOLO [7] the most common detection approaches used two-stage pipelines. The first stage was responsible for generation of the box, the other one for classification. It allowed to archive high accuracy, but kept these models slow. The YOLO series introduced a single-one stage detector - a single convolutional model predicts bounding boxes and class probabilities in one pass over the image. It speeds up the inference and allows YOLO to compete among Real-Time Inference models.

Among 11 versions of YOLO models (see Table [20]), the most important ones are v5, v8, v10 and v11. In some sense, v5, introduced by Ultralytics team, created a foundation for other versions. It came out in PyTorch implementation, which made it easy to deploy, it had multiple sizes (s, m, l, x), so was is easy to adjust to the specific task and constraints. It also became possible to export model to formats for mobile deployment (such as ONNX). Version 8 (see the weight structure in Fig. 2.3) focused more on improving model performance and flexibility. Ultralytics team mainly improved the backbone. The model adopted anchor-free detection: it predicts box coordinates and class probabil-

Table 2.3: Comparisons of RT-DETR versus YOLOv8 and YOLOv10 models. Latency^f denotes the latency of the forward pass without post-processing.

Model	**#Param.(M)**	**FLOPs(G)**	**AP^{val} (%)**	**Latency (ms)**	**Latency^f (ms)**
YOLOv8-N [21]	3.2	8.7	37.3	6.16	1.77
YOLOv10-N [22]	**2.3**	**6.7**	**38.5/39.5†**	**1.84**	**1.79**
YOLOv8-S [21]	11.2	28.6	44.9	7.07	2.33
RT-DETR-R18 [23]	20.0	60.0	46.5	4.58	4.49
YOLOv10-S [22]	**7.2**	**21.6**	**46.3/46.8†**	**2.49**	**2.39**

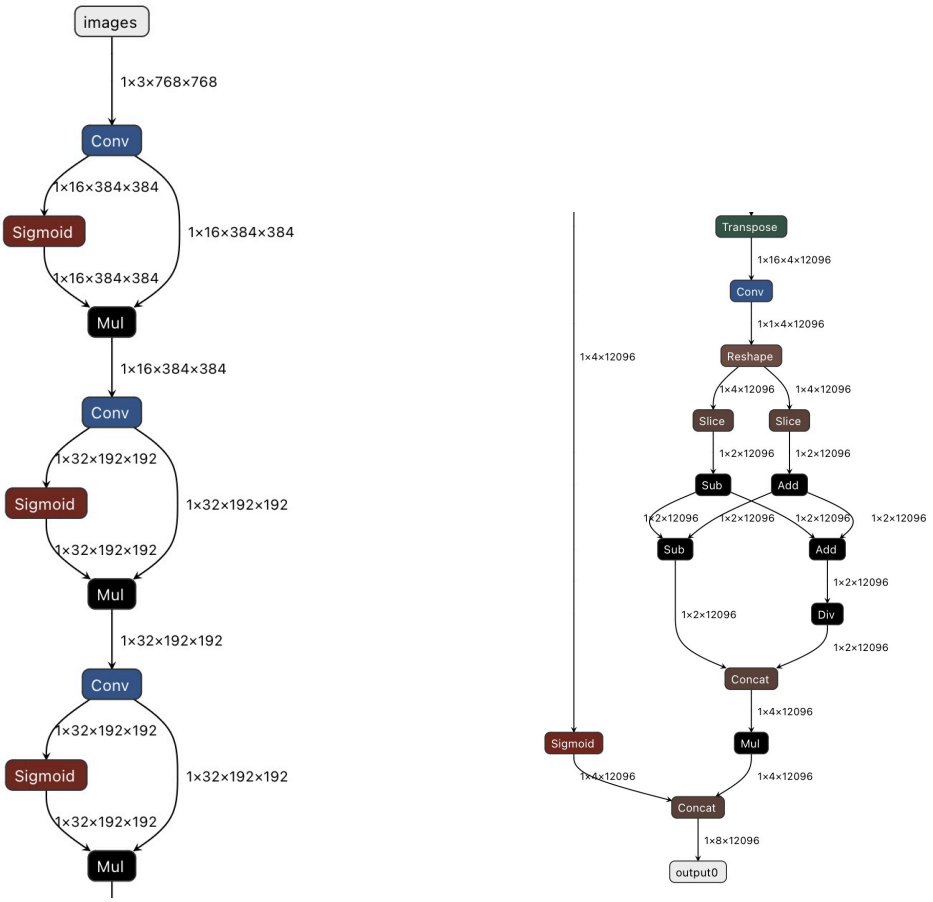
ities without predefined anchor boxes. It simplified the training process and improved its generalization. Version 10 released in 2024 by a Tsinghua University group introduced an important strategy for the box deduplication. Normally YOLO models need an NMS post-process algorithm to remove duplicate detections of the same object, but version 10 presented NMS-free detection. It uses a dual-label assignment during the training process, so the model predicts only one high-confidence box per object. In version 10 large-kernel convolutions and partial self-attention also were introduced to increase the receptive field without heavy computations. The results were impressive. Yolov10-s model reported 1.8 times faster inference compared to its competitor - RT-DETR model with similar AP, while having 2.8 times fewer parameters (see Table 2.3).

The last published version - YOLO11 - didn't introduce significant changes in the architecture structure and is mostly based on YOLOv8 design. For example, it still used NMS to remove duplicated boxes. Authors were focused on optimising the network and achieving higher accuracy per parameter. It jumps in accuracy and efficiency over YOLOv8 and shows higher mAp with fewer parameters.

There also exist some other successful approaches in real-time detection. One of the earlier mentioned models - RT-DETR [23] - uses a Transformer-based architecture. By using transformer decoder, it removes NMS entirely, as was done in YOLOv10 too. It allows RT-DETR model to perform excellent on GPU by efficiently utilizing transformers parallelization. However, YOLO models still remain more parameter-efficient and faster on low-power device or CPUs.

The other well suitable for the Edge detection model is NanoDet [24]. It targets fast CPU inference and deployment on edge devices. It can be as small as 0.95 million parameters, which is smaller than YOLOv8n and Yolov11n models. Authours report a quantized INT8 NanoDet model having approximately 97 FPS on a mobile CPU. In spite of the speed, the model is notably less accurate, having 34% mAP with YOLOv8n having 37% and YOLOv11n - 39.5%.

Summary of model comparison: For edge hardware with limited or no GPU,



(a) Start of the graph (b) End of the graph

Figure 2.3: ONNX representation of YOLOv8 layers.

YOLOv11-n or NanoDet would be preferred due to their small size (with YOLO offering higher accuracy at a bit larger size). With a GPU or NPU, YOLOv11-s or even RT-DETR can be considered.

2.4 Hailo Compiler

Raspberry Pi 5 supports integration with the Hailo-8 accelerator. To deploy neural networks on Hailo devices, the Hailo Dataflow Compiler [25] is required. The final result of the compiler is a binary file `.hef`. The standard DataFlow for this conversion consists of the three main steps:

- Translation of the source model to Hailo model;
- Optimization of model parameters;
- Compilation process itself;

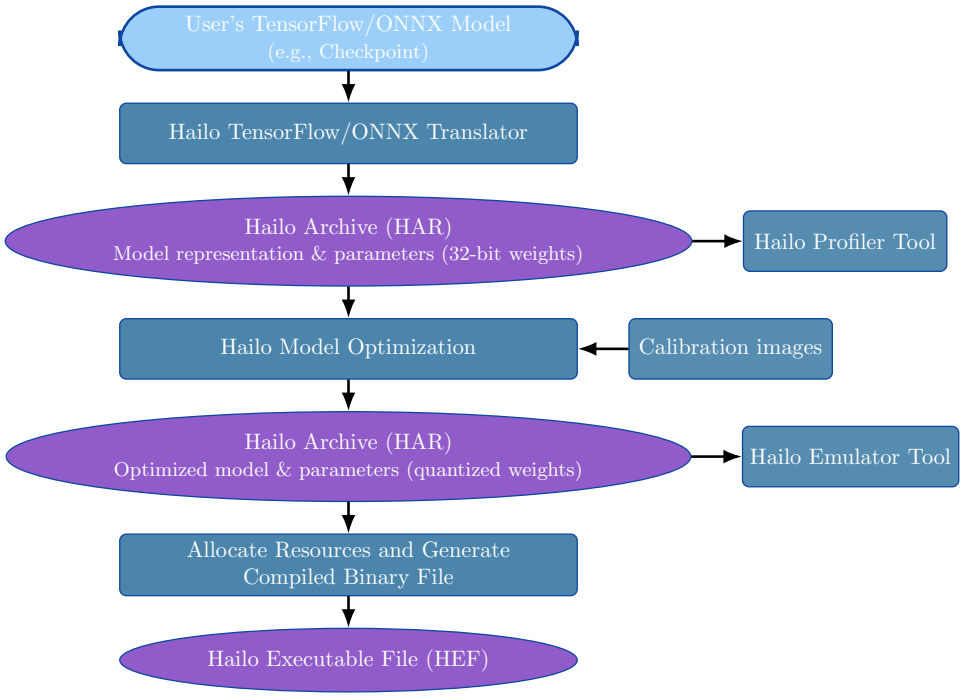


Figure 2.4: Build Process [25]

Hailo can take as an input a user model in ONNX format. ONNX is an open format developed to unify ML models for a variety of frameworks and compilers by defining a common set of operators and represent models via graph structure.

The first step in the Figure 2.4 is a translation from the user's model to the `.har` representation. The Hailo Translation API parses user's model and produces a `.har` (Hailo ARchive).

- HAR is a zip package that contains:
  - HN— "Hailo Network": a JSON graph of the network (layers, tensor shapes/d-types, connections, metadata).
  - NPZ—NumPy arrays with the floating-point weights(and sometimes other tensors like batch normalization params).

This step is needed, so parameters of the model are normalized from the other possible checkpoint configurations (used operation and attributes names, FP32 weights), and the optimizer has a clean input.

The second step is a model optimization. The key part of the optmization is the conversion of the model from FP32 to INT8. It is done with an additional run of calibrarion module.

There exist different ways to quantize a model. To my knowledge, Hailo Compiler is using the Linear Quantization:

Key benefits from this approach is its efficiency, support by major DL frameworks and easy implementation [26]. However, the drawback is a visible precision loss as many nearby floating-point values collapse to the same quantized level. Layers with large dynamic range (e.g., softmax) are more sensitive to this method.

Assume an input floating-point value, the linear quantization is

$$q = \text{round}\Big(Z + \frac{r}{S}\Big).$$

Where

- r: original 32-bit floating-point number
- q: quantized representation (integer)
- S: scale factor (step size)
- Z: zero point (integer), which allows $r = 0$ to be represented exactly by the integer Z

Dequantization is defined by the formula:

$$r \approx (q - Z)\, S.$$

It is also illustrated at the Figure 2.5

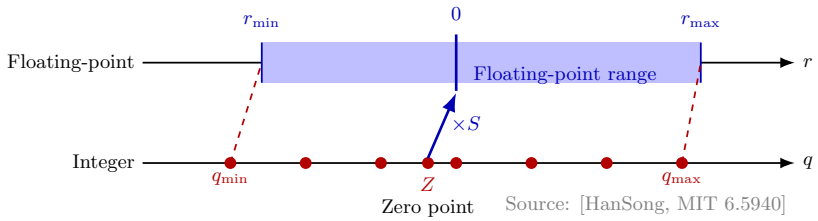


Figure 2.5: Build Process [26]

In order to perform this optimization, the calibration set of raw, unlabeled samples is gathered. It should match a real input distribution (color space, normalization). Compiler uses this set to run a neural network on it and to log per-tensor activation distributions. So, it records min/max values and percentiles. By doing so, an optimizer gets r_{min} and r_{max} values. Which are used to compute q_{min}, q_{max} and Z - point (see Figure 2.5). It is important to do so per tensor as different layers produce values with very different ranges/outliers. Choosing ranges from their actual distributions minimizes quantization error.

The last step is a compilation itself, when model is compiled to a binary file `.hef`. The compiler is responsible for allocation hardware resources, so the highest possible FPS can be archived. It performs allocation by mapping neural network layers onto the hardware [27] (compute elements, memory elements, and control elements) and schedule their execution under the chip's resource constraints. If the network is too large, the compiler will partition the network into several contexts, allowing bigger models still run with some performance overhead. The compiler might tile large tensors, split layers into smaller pieces to fit into memory and to utilize parallelism. Key strategy of this hardware mapping is to perform as many computation as possible (possibly for difference layers) on the kept on the chip memory tensors. As part of scheduling, compiler also defines the data flow from layer to layer, chaining producers and consumers layer clusters. For example, while the layer i is computed on one side of the chip, another part of the chip might start executing layer i+1, or it can perform several different operations in parallel of the same fetched from the memory piece of data (Fig. 2.6). This schedule is constructed during the compilation step. Apart from the execution plan, compiler also performs on-chip Non-Maximum Suppression (NMS) fusion for models like YOLO. NMS algorithm performs deduplication of several predicted for the same object boxes. For instance, the screw could be with different probabilities be predicted both as machine and wood screw. So, the post-processing NMS is performed to keep only the most suitable box. Hailo's compiler can inregrate NMS as the final layer of the network, so it runs on the hailo itself and not on the CPU after the inference. It improves the performance not only due to

localization of computation, but also due to reduced data transfer - only filtered detections are sent to the Raspberry Pi.

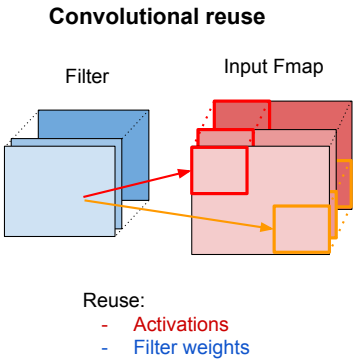


Figure 2.6: Example of convolutional reuse. As a filter slides over the input feature map, large portions of the input activations (red/orange rectangles) and the filter weights (blue blocks) are repeatedly reused for neighbouring convolution windows. Because adjacent receptive fields overlap, only a small part of the data must be fetched again, enabling significant savings in memory access and energy consumption. (Inspired by [28])

The final Hailo Executable File contains:

- Low-level program for the target device (layer schedule, the memory buffer addresses/sizes, instruction sequences etc.)
- Weights and biases
- Metadata and calibration information

2.5 Summary of the Literature Study

The literature review establishes the technical context and design space for real-time object detection on constrained edge hardware (Raspberry Pi 5) with optional NPU acceleration (Hailo-8). We surveyed (i) detection formulations and annotation strategies, (ii) dataset options and label formats, (iii) lightweight detector families suitable for edge deployment, and (iv) compilation/quantization toolchains required to execute modern models on dedicated accelerators.

Detection task & annotations. Object detection can be localized with axis-aligned boxes (AABB), oriented boxes (OBB), polygons, or pixel masks. While OBB/polygons align better with prolongated parts (e.g., screws), AABB remains the best choice for transfer learning with common pretrained weights (e.g., COCO) and for real-time deployment. The review also summarizes conversions from corner points and the practical AABB enclosure of OBBs, together with format mappings to COCO JSON and YOLO TXT. This motivates our use of AABB for the fastener domain to minimize engineering.

Datasets. COCO is the dominant pretraining source for AABB detectors and ensures strong generalization, whereas DOTA provides OBB `*-obb` variants. A domain-specific fastener set (MVTec Screws) offers OBB labels but is limited in scale. Due to the fact that public datasets rarely contain fasteners, the study supports a compact, grayscale, task-specific dataset and to perform transfer learning from pretrained on COCO weights.

Model families for edge. Single-stage YOLO variants remain the most practical for low-power deployments due to accuracy–latency trade-offs. In the past years YOLO evolved toward anchor-free heads, improved backbones, and—in v10—NMS-free training; YOLOv11 focuses on higher accuracy per parameter and streamlined exports. Transformer-based RT-DETR remover NMS and scales well on GPUs, but it is typically more compute-expensive on CPUs/edge. Ultra-small models like NanoDet can reach very high CPU FPS with INT8 quantization, at the expense of accuracy. Overall, the review motivates evaluating YOLO -nano/-small variants first for Raspberry Pi CPUs, with RT-DETR as a comparative reference when acceleration or higher latency budgets exist.

Compilation and quantization. Executing modern detectors on an NPU requires a specific toolchain. The Hailo flow (ONNX $\rightarrow$ HAR $\rightarrow$ HEF) translates graphs, calibrates activations, and performs graph/resource scheduling. The study explains linear INT8 PTQ (scale/zero-point, per-tensor ranges from calibration statistics), YOLO post-process/NMS, and context partitioning when models exceed on-chip resources. These mechanisms are important for moving the entire detection path (tensor in $\rightarrow$ filtered detections out) onto the accelerator, minimizing CPU load and PCIe/host transfers.

ONNX as an interchange. ONNX is identified as the stable intermediate for exporting PyTorch checkpoints and enabling downstream compilation. Its operator coverage and ecosystem support reduce model operations variability across training and deployment.

Implications for this project.

- **Labeling choice:** We will use AABB to align with pretrained YOLO checkpoints and simplify the Hailo export pipeline. However, the use of OBB is still posible and

might improve the quality of the detection.

- **Model shortlist:** We prioritize YOLOv8/10/11 -nano/-small for Raspberry Pi-only baselines. We also include RT-DETR-R18 - YOLO competitor, for transformer reference and for the speed/quality tradeoff experiments.
- **Tooling:** Standardize on PyTorch → ONNX and the Hailo compiler with INT8 PTQ using a calibration set that matches deployment preprocessing.
- **Performance strategy:** For strict real-time throughput,we run experiments on Hailo-8 with post-processing

Gaps and risks. OBB supervision could further improve fine orientation for screws but would require new pretraining or costly relabeling. Quantization sensitivity on small, fine-grained targets remains a risk. Careful calibration selection and post-training checks are necessary. Finally, runtime on Raspberry Pi pipelines is latency-limited, so the use of accelerator is crucial for real-time performance.

Table 2.4: Summary of YOLO releases [20]

Release	**Year**	**Tasks**	**Contributions**	**Framework**
YOLO	2015	Object Detection, Basic Classification	Single-stage object detector	Darknet
YOLOv2	2016	Object Detection, Improved Classification	Multi-scale training, dimension clustering	Darknet
YOLOv3	2018	Object Detection, Multi-scale Detection	SPP block, Darknet-53 backbone	Darknet
YOLOv4	2020	Object Detection, Basic Object Tracking	Mish activation, CSPDarknet-53 backbone	Darknet
YOLOv5	2020	Object Detection, Basic Instance Segmentation (via custom modifications)	Anchor-free detection, SWISH activation, PANet	PyTorch
YOLOv6	2022	Object Detection, Instance Segmentation	Self-attention, anchor-free OD	PyTorch
YOLOv7	2022	Object Detection, Object Tracking, Instance Segmentation	Transformers, E-ELAN reparameterisation	PyTorch
YOLOv8	2023	Object Detection, Instance Segmentation, Panoptic Segmentation, Keypoint Estimation	GANs, anchor-free detection	PyTorch
YOLOv9	2024	Object Detection, Instance Segmentation	PGI and GELAN	PyTorch
YOLOv10	2024	Object Detection	Consistent dual assignments for NMS-free training	PyTorch
YOLOv11	2024	Object Detection; (variants for) Instance Segmentation, Pose, OBB	Updated training pipeline and architectures; better accuracy–speed trade-offs; streamlined exports (ONNX/TensorRT) in Ultralytics	PyTorch

3 Theory

Metrics description

Metrics description:

- **Intersection over Union (IoU)**:
  For a predicted box p and ground-truth box g, the IoU formula is:

$$\text{IoU}(p, g) = \frac{|p \cap g|}{|p \cup g|}$$

  IoU plays role of evaluating an object localization.
  It quantifies the overlap between a prediction and ground truth bounding boxes by calculating the ratio of the intersection (overlapping area) to the union (total area covered by both boxes). IoU is a building block for calculating mAP.
  The resulting score is a value between 0 and 1, which quantifies how accurately a model has located an object in an image. A score of 1 represents a perfect match, while a score of 0 indicates no overlap at all.
  The key part of using IoU is to define *"IoU threshold"*. This threshold is a predefined value (e.g., 0.5) that determines whether a prediction is correct. If the IoU score for a predicted box is above this threshold, it is classified as a "true positive" (TP). If the score is below, it's a "false positive" (FP).
  This threshold directly influences other performance metrics like Precision and Recall.

- **Precision and Recall**:
  Precision defines the proportion of true positives among all positive predictions, measuring the ability to avoid misclassification.

$$Precision(P) = \frac{\text{TP}}{\text{TP} + \text{FP}},$$

  On the other hand, Recall calculates the proportion of true positives among all actual positives, measuring the model's ability to detect all instances of a class.

$$Recall(R) = \frac{\text{TP}}{\text{TP} + \text{FN}},$$

  where
  TP - True Positive, FP - False Positive, TN - True Negative and FN - False Negative

predictions.

- **Average Precision (AP)**:
  AP computes the area under the precision-recall curve [4], providing a single value that encapsulates the model's precision and recall performance.

$$\mathrm{AP}_\tau = \int_0^1 p(r;\tau)\,dr,$$

  In order to build the precision-recall curve, the score (confidence) threshold (t) is used:

  – Every detection has a score $s \in [0, 1]$.
  – We sweep t from high $\rightarrow$ low (equivalently sort detections by score desc and add them one-by-one).
  – At each t we keep detections with $s \geq t$ and recompute P/R.
  – This sweep produces the P–R curve.
  – AP is the area under this P–R curve;

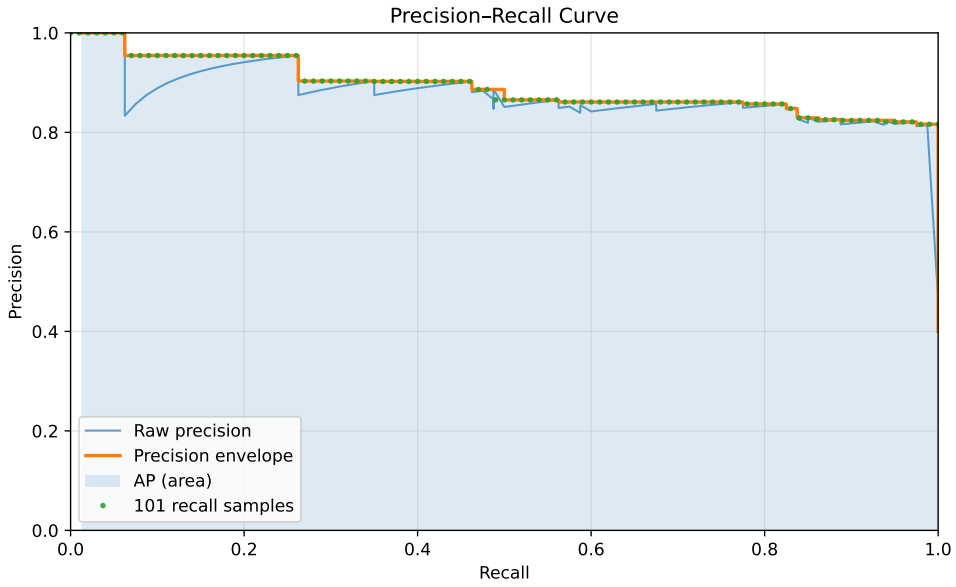


Figure 3.1: Precision–Recall curve.

- **Mean Average Precision (mAP)**:
  Another metric - mAP, extends the concept of AP by calculating the average AP values across multiple object classes.
  This is useful in multi-class object detection scenarios to provide a comprehensive

evaluation of the model's performance.

$$\mathrm{mAP}_{0.50} = \frac{1}{C} \sum_{c=1}^{C} \mathrm{AP}_{c,\,0.50},$$

$$\mathrm{mAP}_{[0.50:0.95]} = \frac{1}{C} \sum_{c=1}^{C} \frac{1}{10} \sum_{k=0}^{9} \mathrm{AP}_{c,\,(0.50+0.05k)}.$$

The parameter C is the number of classes for which detection and classification are performed.

The subscript 0.5, as well as the set [0.50:0.95], denotes the IoU threshold: predictions with IoU ≥ 0.5 are counted as true positives (TP).

4 Methodology

The worked is performed in 6 steps:

- **Modeling**
  - Defining the set of suitable for Edge AI models;
  - Exploration theirs constraints and performance;
- **Annotation & Dataset**
  - Camera SDK setup and programming;
  - Define annotation format and tools;
  - Perform annotation and prepare suitable for the chosen model dataset;
- **Training**
  - Perform a Transfer Learning Training on a GPU;
  - Defining the pretrained weights based on the task and required annotation quality;
- **Quantization & Compilation**
  - Configuration of a Hailo Compiler for the Hailo-8 AI Accelerator deployment;
  - INT8 PTQ with calibration set; Export .onnx $\rightarrow$ .har $\rightarrow$ .hef;
- **Edge Pipeline**
  - Programming of the Camera Real-Time Streaming;
  - Perform Real-Time Inference;
- **Benchmarking**
  - CPU vs Hailo-8 AI comparison;
  - FPS and Detection Metrics measures;
  - Model Performance Comparison;

5 Proposed Solution / Design

5.1 Dataset and Annotation

The dataset consists of 2131 images (see Fig. 5.1), which are split into 1983 images for the training, 98 images for the validation and 50 images for the test. Images have a 960x960 size and are grayscale.

The augmentation includes:

Table 5.1: Data augmentation settings.

Parameter	Value
Outputs per training example	3
Flip	Horizontal, vertical
90° rotation	Clockwise, counter-clockwise, upside down
Hue	Between −24° and +24°
Saturation	Between −25% and +25%
Brightness	Between −22% and +22%
Exposure	Between −10% and +10%
Blur	Up to 2.5 px
Noise	Up to 1.72% of pixels

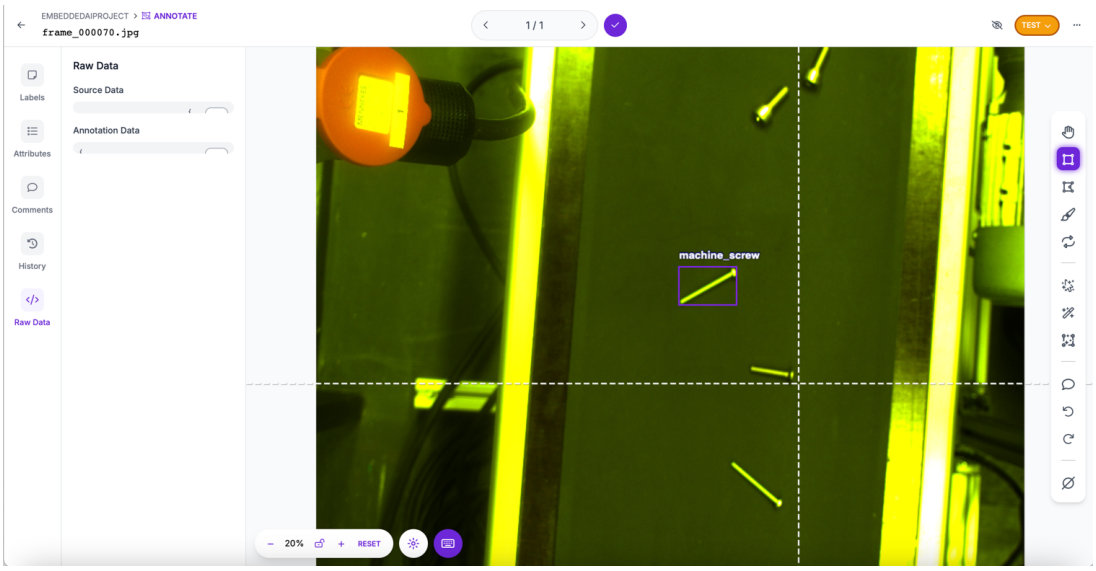


Figure 5.1: Roboflow Annotation Illustration

Annotation was performed with the use of the Roboflow Framework [28]. It is a platform for computer vision datasets and models. It provides an API for image upload, fast conversion between formats (e.g., YOLO, COCO), in-browser annotation and labeling for bounding boxes, segmentation and classification tasks.

The link to the dataset placed at the Roboflow platform is available in the reference section [29]

5.2 Camera Setup

The camera used in the project is IDS uEye UI-3590CP-C-HQ Rev.2. It is an industrial machine-vision camera from USB3 uEye Cp series. IDS provides a uEye SDK, though which the control of streaming, configuration, exposure/gain, etc. is happening.

Figure 5.2: The picture of the camera is taken from the IDS manual [30]

The parameters logged while to capturing frames for the further annotation:

Parameter	Value
FPS	7.0
Exposure	80 ms
Width	4912 px
Height	3684 px
Color	true
Frames captured	70
Frames saved	70
Gain	10%
Gamma (×100)	130

To enable real-time image streaming, the pyueye Python wrapper for IDS cameras [31] was used. This library provides a low-level interface to the uEye SDK and requires a basic understanding of several core concepts, which have to be configured for the Streaming:

Core concepts:

Camera handle The object `hCam = ueye.HIDS(0)` represents a camera identifier. It behaves as an integer-like handle and must be passed to almost every SDK function.

Image memory Image buffers are allocated on the camera side, and the driver keeps track of which buffer is currently active.

AOI (Area of Interest) Defines the rectangular region of the sensor to capture. It is specified by an offset (top-left position) and the desired width and height.

Color mode / pixel format Specifies how pixels are encoded, such as `UEYE_RGB8_PACKED` or `UEYE_MONO8`.

5.3 Hardware

Among the most popular development platforms is a Raspberry Pi family, which offers a flexible setting for deploying lightweight AI systems. Raspberry Pi 5 used for this work includes:

An additional hardware used during the experiment is a Hailo-8 accelerator offering 26 TOPS inferencing performance respectively.

Table 5.2: Raspberry Pi 5 specifications

Compute	
SoC	2.4 GHz quad-core 64-bit Arm Cortex-A76 (crypto ext.) 512 KB L2 per core 2 MB shared L3
GPU	VideoCore VII; OpenGL ES 3.1, Vulkan 1.3
Memory and Storage	
RAM	8 GB
microSD	64 GB
Performance (theoretical)	
Estimated FP32 peak	76.8 GFLOPs (2.4 GHz)

The hardware setup was chosen to be representative of realistic edge deployments. The Raspberry Pi serves as a low-power, widely available ARM platform that can both run the models on its own CPU and act as a host controller for the external accelerator. The Hailo-8 NPU, attached via PCIe, represents a class of specialized inference accelerators increasingly used in industrial and embedded systems to meet strict real-time and power constraints. By combining these two, we can evaluate three aspects in a controlled way: how far we can go with a CPU-only system, how much additional performance and efficiency is gained by using an NPU, and what trade-offs this introduces in terms of compilation, deployment complexity, and hardware cost.

The YOLOv8 and YOLOv11 are both provided by Ultralytics as a PyTorch checkout. In order to use it on accelerator, this checkout has to be converted to a suitable format by Hailo Compiler.

Model training and evaluation were conducted on a host machine provided by NTNU, equipped with an AMD Ryzen 9 5900X CPU and an NVIDIA GeForce RTX 3070 GPU. Their detailed specifications are presented in Tables 5.4 and 5.3.

Table 5.3: GPU specifications: NVIDIA GeForce RTX 3070 (Ampere, GA104)

Property	**Value**
Architecture	Ampere (GA104)
CUDA cores	5,888
Tensor cores (3rd gen)	184
RT cores (2nd gen)	46
Base / Boost clock	1.5 GHz / 1.73 GHz
Memory (VRAM)	8 GB GDDR6
Memory bus width	256-bit
Memory bandwidth	∼448 GB/s
CUDA version	12.4
FP32 (single precision)	∼20 TFLOPs (peak)
Form factor	PCIe 4.0 ×16

Table 5.4: Host CPU specifications: AMD Ryzen 9 5900X

Property	Value
Architecture	x86_64
CPU op-modes	32-bit, 64-bit
Address sizes	48-bit physical, 48-bit virtual
CPU(s)	24 (threads)
Cores / Threads / Sockets	12 / 2 / 1
Vendor	AuthenticAMD
Model name	AMD Ryzen 9 5900X 12-Core Processor
Family / Model / Stepping	25 / 33 / 2
Frequency boost	enabled
Max / Min freq	4950.2 MHz / 2200.0 MHz
BogoMIPS	7385.80
L1d / L1i (sum)	384 KiB / 384 KiB (12 inst.)
L2 / L3 (sum)	6 MiB (12 inst.) / 64 MiB (2 inst.)
Estimated FLOPS	0.71 TFLOPs (FP32 peak 3.7 GHz)

5.4 Demonstration

Some demo pictures are listed in Appendix A at the end of the report.

6 Testing and Results

6.1 Experimental Setup

We trained seven models (YOLO v8, v10, v11 -n/-s and RT-DETR-R18) to provide an overview of their performance in terms of accuracy and speed. For the training NVIDIA GeForce RTX 3070 was used, which provides up to 20 TFLOPs of FP32 (single-precision) and has 8GB of RAM. After the training and evaluation on the GPU, models were deployed on Raspberry Pi 5 for the on-device speed benchmarking and demo detection streaming.

Then, models were optimized and compiled to the `.hef` format, so they could be run on a Hailo-8 AI accelerator with 26 TOPS.

On average all models required around 100 epochs of training and default YOLO and RTDERT configutations. Training of YOLO models was performed with the Ultralytics YOLO framework using an input resolution of 960×960 pixels (imgsz=960) and 640×640 pixels (imgsz=640). Each model was trained for up to 100 epochs (epochs=100) with a mini-batch size of 4 (batch=4) on a single GPU (device=0) and 4 data-loader workers (workers=4). To speed up training, images and labels were cached on disk (cache=disk). Mosaic augmentation was disabled during the last 10 epochs (close_mosaic=10) to fine-tune on images that are closer to the real test distribution. A cosine learning-rate schedule was used (cos_lr=True), and early stopping was enabled with a patience of 30 epochs (patience=30), i.e., training stopped if the validation metric did not improve for 30 consecutive epochs.

RT-DETR with a PResNet-18 backbone (RT-DETR-R18) was trained on the custom COCO-format fastener dataset with 5 classes. Images were resized to 640×640 and processed using the standard RT-DETR augmentation pipeline. The backbone is a pretrained PResNet-18 (no frozen stages), followed by a HybridEncoder with three input feature levels (channels [128, 256, 512]) and a hidden dimension of 256 with expansion 0.5. The transformer uses 3 decoder layers and 100 denoising queries. Training ran for 72 epochs with gradient clipping at a maximum L2 norm of 0.1. An exponential moving average (EMA) of the model weights was maintained during training (use_ema=True) using a ModelEMA with decay 0.9999 and a warm-up of 2000 steps. Distributed training was configured with find_unused_parameters=True to avoid issues with unused branches in the model graph.

6.2 Results

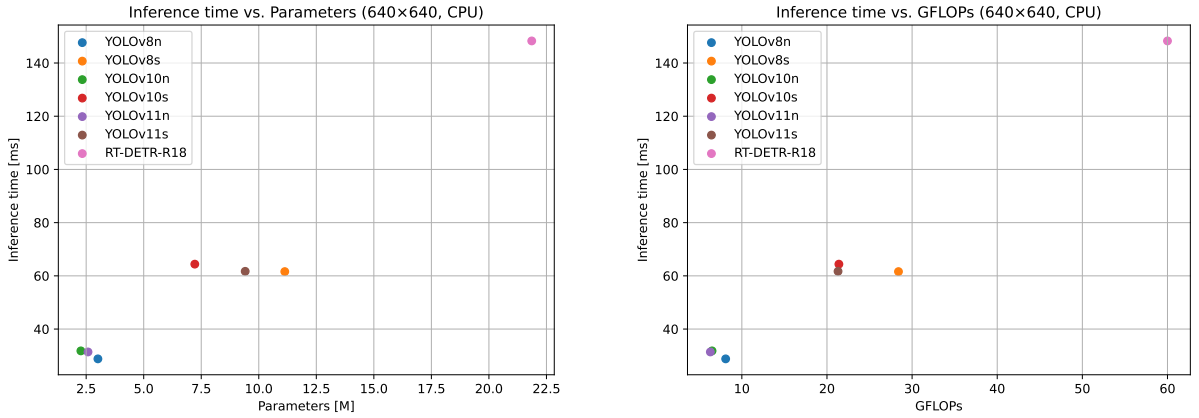
6.2.1 Runtime analysis

Figure 6.1 summarises the runtime characteristics of all evaluated detectors on the screw test video at 640×640 input resolution on host CPU. For all models, the preprocessing and postprocessing stages are relatively cheap, staying around 2 ms and below 0.5 ms per frame respectively, The total latency is dominated by the inference time.

Among the YOLO variants, the nano models (YOLOv8n, YOLOv10n, YOLOv11n) achieve total runtimes between 31.4 ms and 34.3 ms per frame, which is equal to 30 FPS. Despite small differences in parameter count and GFLOPs, their inference times are very similar, with YOLOv8n being slightly faster than YOLOv10n/YOLOv11n. The small models (YOLOv8s, YOLOv10s, YOLOv11s) double the parameter count and GFLOPs compared to their nano version, which leads to a near-doubling of the inference time (about 62–64 ms) and a reduction to approximately 15 FPS.

RT-DETR-R18 shows a significantly different runtime profile. With 21.9 M parameters and an estimated 60 GFLOPs, its inference time of 148.3 ms leads to a total latency of 150.4 ms per frame, i.e. about 6–7 FPS on the same CPU. While preprocessing and postprocessing remain comparable to the YOLO models, the transformer-based backbone and head are substantially more expensive than the convolutional YOLO architectures.

Figure 6.1 highlights a clear trade-off between model complexity and real-time performance, with the nano YOLO variants best suited for latency-constrained deployments on host CPU. Taking into an account the hardware configuration of the host CPU (Table 5.4) and Raspberry Pi 5 CPU (Table 5.2), the scaling factor is $\approxeq 0.71TF/0.0768TF \approxeq 9.25x$ slower on Pi if compute-bound and approximately x3 on practice. Then an estimation of the model performance on Raspberry Pi 5: lands at 1.1 FPS (nano), 0.6 FPS (small) and 0.24 FPS (RT-DETR).



(a) Inference time vs. number of parameters.

(b) Inference time vs. GFLOPs.

Figure 6.1: Comparison of trained models in terms of inference time, model size, and computational cost at 640×640 input resolution.

6.2.2 Performance overview

As shown in the runtime overview subsection, the YOLO models demonstrated the best inference performance, particularly the -nano variants. In this subsection, only the -nano versions of YOLO are compared against each other, focusing on their runtimes when executed on the Raspberry Pi 5 CPU and their mAP performance.

Table 6.1 compares the validation performance of the three nano models on the screw validation set v8n, v10n and v11n. Overall, all architectures achieve strong detection quality, with class $\text{mAP}_{50\text{–}95}$ values between 0.728 and 0.742. YOLO8n attains the highest mAP_{50} (0.964) and competitive $\text{mAP}_{50\text{–}95}$ (0.733), while YOLO11n slightly improves the overall $\text{mAP}_{50\text{–}95}$ to 0.742 at the cost of a marginal drop in precision (0.920 vs. 0.927). YOLO10n sits in between, with the best mAP_{50} (0.969) but a slightly lower $\text{mAP}_{50\text{–}95}$ (0.728).

Across classes, all three models perform best on nut, washer and wood_screw, where precision and recall are above 0.95 and $\text{mAP}_{50\text{–}95}$ reaches 0.76–0.91. In terms of runtime, all nano models show very similar behaviour on the Raspberry Pi 5, with preprocessing around 8-9 ms and inference between approximately 1.0 and 1.1 s per image. The differences in latency (YOLO8n being slightly faster than YOLO10n and YOLO11n) are small compared to the overall inference time, so model choice is primarily driven by the modest accuracy differences rather than runtime on this platform.

From the comparison of these three models it seems that v8n is slightly better than

Table 6.1: Validation summaries.

(a) YOLO8n [32]

Class	Images	Instances	Box(P)	R	mAP_{50}	$mAP_{50–95}$
all	98	363	0.927	0.951	0.964	0.733
machine_screw	25	123	0.931	0.967	0.989	0.625
miscellaneous	14	16	0.754	0.812	0.845	0.562
nut	16	41	0.988	1.000	0.995	0.777
washer	20	43	1.000	0.991	0.995	0.906
wood_screw	37	140	0.960	0.986	0.994	0.794

Speed per image (Raspberry Pi 5): preprocess 8.3 ms; inference 1018.4 ms; loss 0.0 ms; postprocess 1.2 ms.

(b) YOLO10n

Class	Images	Instances	Box(P)	R	mAP_{50}	$mAP_{50–95}$
all	98	363	0.924	0.920	0.969	0.728
machine_screw	25	123	0.917	0.893	0.935	0.584
miscellaneous	14	16	0.921	0.729	0.933	0.602
nut	16	41	0.962	1.000	0.995	0.787
washer	20	43	0.957	1.000	0.995	0.901
wood_screw	37	140	0.863	0.979	0.986	0.764

Speed per image (Raspberry Pi 5): preprocess 8.0 ms; inference 1033.1 ms; loss 0.0 ms; postprocess 0.3 ms.

(c) YOLO11n

Class	Images	Instances	Box(P)	R	mAP_{50}	$mAP_{50–95}$
all	98	363	0.920	0.933	0.961	0.742
machine_screw	25	123	0.842	0.927	0.945	0.609
miscellaneous	14	16	0.922	0.739	0.889	0.620
nut	16	41	0.969	1.000	0.995	0.799
washer	20	43	0.995	1.000	0.995	0.908
wood_screw	37	140	0.872	1.000	0.983	0.772

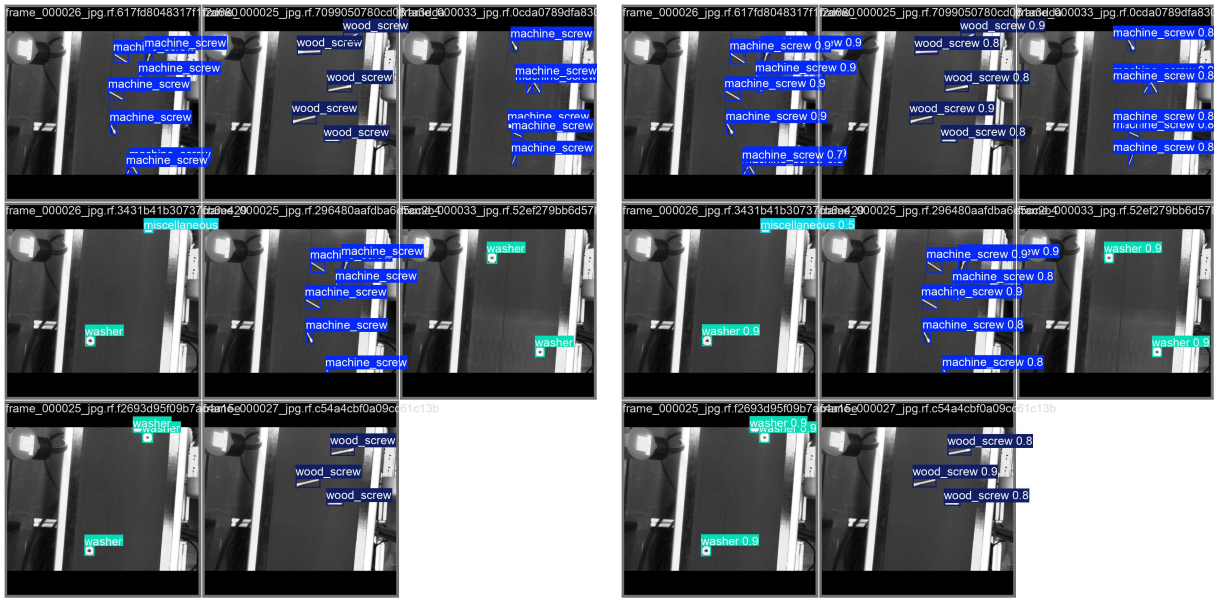
Speed per image (Raspberry Pi 5): preprocess 8.8 ms; inference 1104.2 ms; loss 0.0 ms; postprocess 1.4 ms.

v10 and v11, as it shows both the highest mAP_{50} and the lowest inference time, however, models v10 and v11 have good trade-off between size and accuracy.

6.2.3 Prediction Examples

Figure 6.2 illustrates an example of YOLOv8n predictions obtained from the PyTorch checkpoint. For convenience of the presentation each image contains objects belonging to a single class only. As we can see, all objects are correctly classified. However, the confidence scores varies across classes. For example, prediction of the miscellaneous class

has the lowest confidence score. It happens due to the limited amount of such objects in the training dataset. Objects of this class are also much more diverse, since screws, nuts, and washers can be classifyed as miscellaneous if only a partial view of these objects is present at the picture.



(a) Ground truth: yolov8n

(b) Prediction: yolov8n

Figure 6.2: Ground truth and prediction for `yolov8n`.

6.2.4 Image size influence on model performance

Table 6.2 compares YOLO11n evaluated on the screw test set at two input resolutions, 640×640 and 960×960. Overall, the model achieves very similar performance at both scales: the $mAP_{50–95}$ stays mostly unchanged (0.737 vs. 0.740), while precision and recall slightly improve at 960×960 (from 0.930/0.922 to 0.938/0.963). For most classes, mAP_{50} and $mAP_{50–95}$ either increase (e.g. nut and wood_screw) or stay almost identical (washer). The miscellaneous class remains the most challenging: recall benefits from the higher resolution (0.687 to 0.846), but its $mAP_{50–95}$ slightly decreases (0.522 to 0.516), indicating that additional detections are partly offset by noisier localization or extra false positives.

The normalized confusion matrices in Fig.6.3 and Fig.6.4 provide a qualitative view of these effects. At 640×640 (Fig. 6.3), most probability mass already lies on the diagonal, with confusion between machine_screw and miscellaneous, and between wood_screw and the background. At 960×960 (Fig.6.4), the diagonal entries for machine_screw, nut, washer, and wood_screw become slightly stronger, reflecting the small gains reported in Table 6.2. However, the miscellaneous class continues to show the largest off-diagonal mass, confirming that it is the main source of residual confusion.

Table 6.2: YOLO11n test performance on the screw dataset for two input resolutions.

	640×640				960×960			
Class	P	R	mAP_{50}	$mAP_{50–95}$	P	R	mAP_{50}	$mAP_{50–95}$
all	0.930	0.922	0.944	0.737	0.938	0.963	0.952	0.740
machine_screw	0.954	0.938	0.988	0.709	0.965	0.969	0.990	0.711
miscellaneous	0.899	0.687	0.775	0.522	0.916	0.846	0.789	0.516
nut	0.922	1.000	0.988	0.765	0.971	1.000	0.995	0.787
washer	0.942	1.000	0.995	0.895	0.917	1.000	0.995	0.894
wood_screw	0.931	0.985	0.973	0.791	0.921	1.000	0.991	0.793

Finally, the validation log at 960 × 960 reports an average CPU inference time of 124.8 ms per image (with 4.2 ms for preprocessing and 0.5 ms for postprocessing), i.e. roughly 8 FPS. Given the marginal accuracy gains compared to 640 × 640, this highlights a clear resolution - speed trade-off: higher input resolution yields slightly more stable detections on most classes, but at a significant computational cost on the target hardware.

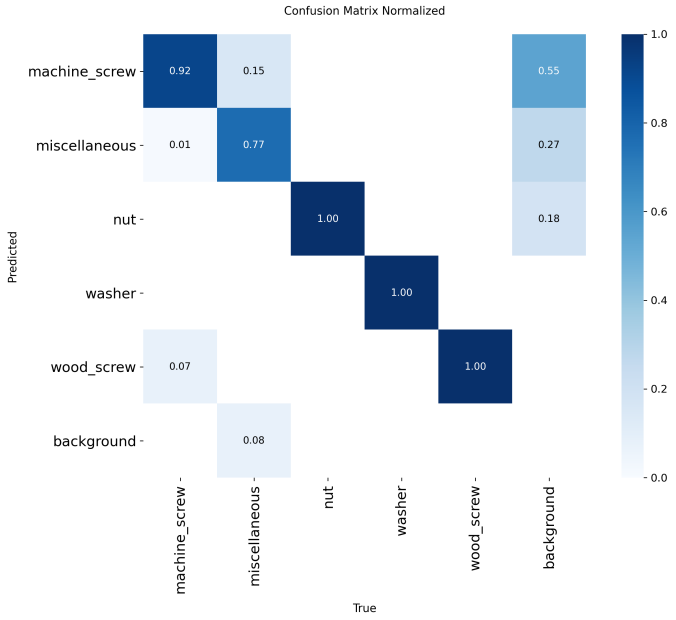


Figure 6.3: YOLO11n confusion matrix at 640 × 640.

6.2.5 Hailo impact

The impact of offloading inference to the Hailo-8 accelerator is significant. While the Raspberry Pi 5 CPU executes the nano models at around 1s per image (see. Table 6.1), the Hailo benchmark for the compiled `yolov8n.hef` reports over 140 FPS in both HW-only and streaming modes, with a hardware latency of only 10.8 ms per frame (Table 6.3). This corresponds to more than two orders of magnitude improvement in throughput com-

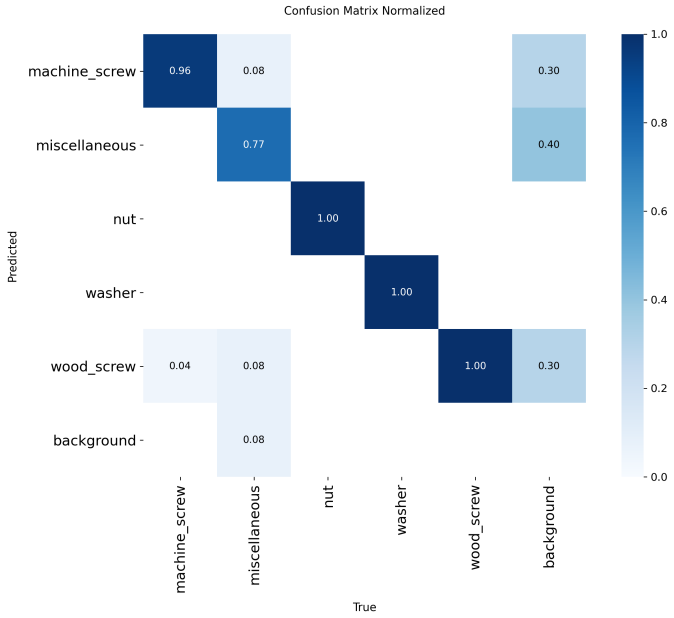


Figure 6.4: YOLO11n confusion matrix at 960×960.

pared to pure CPU execution. The HEF metadata in Table 6.4 confirms that the network is mapped to a single `yolov8n` context on the HAILO8 architecture, while Tables 6.5 and 6.6 show that both the input tensor and the YOLOv8-specific NMS post-processing (including class-aware suppression and bounding-box limits) are integrated into the accelerator pipeline. This means that the entire detection stage from raw image tensor to filtered detections can be executed on the Hailo device with minimal CPU involvement, enabling real-time operation on the conveyor setup where the Raspberry Pi 5 alone is clearly insufficient. The YOLOv10 and YOLOv11 models (both -n and -s) require a 3-context mapping on Hailo-8 to satisfy resource constraints while preserving performance. The YOLOv8-s and -n models compiled significantly faster than newer versions because the Hailo compiler was able to place the entire network in a single context. This removes the expensive multi-context partitioning search and reduced the number of mapping iterations (28 vs. 170 for YOLO11-n), resulting in a total mapping time of 47 seconds and compilation time of 22 seconds. For YOLO10s it takes around 10 minutes to partition to 3 context. The drop in FPS from YOLOv8 to YOLOv10 and YOLOv11 is more than two times. However, even the lowest throughput (approximately 30 FPS on YOLOv11s) remains sufficient for real-time detection on a conveyor belt moving at 2 cm/s. At 30 FPS, each inference takes about 33 ms, while the camera—reaching a maximum of around 7 FPS—produces a new frame every 143 ms.

This means the detector processes frames over 4x faster than they are captured, ensuring that inference is never the bottleneck. At 2 cm/s, the belt moves only about 0.66 mm during a 33 ms inference window, which is negligible relative to the object sizes, confirming that the system comfortably operates in real time.

Table 6.3: Combined Hailo benchmark results for all YOLO models.

Model	FPS (HW-only)	FPS (Streaming)	Latency (HW)
yolov8n	142.371	142.370	10.796 ms
yolov8s	82.70	82.70	20.60 ms
yolov10n	64.39	64.38	13.61 ms
yolov10s	35.79	35.79	25.38 ms
yolov11n	57.39	57.38	15.20 ms
yolov11s	30.25	30.25	28.89 ms

Table 6.4: HEF metadata (`yolov8n.hef`).

Field	Value
Architecture	HAILO8
Network group	`yolov8n` (Single Context)
Network	`yolov8n/yolov8n`

Table 6.5: VStream infos.

Direction	Name	Spec
Input	`yolov8n/input_layer1`	`UINT8, NHWC(960×960×3)`
Output	`yolov8n/yolov8_nms_postprocess`	`FLOAT32, HAILO NMS BY CLASS` (number of classes: 5, max bboxes/class: 100, max frame size: 10020)

Table 6.6: Post-process (`YOLOV8-Post-Process`) parameters.

Parameter	Value
Op / Name	`YOLOV8 / YOLOV8-Post-Process`
Score threshold	0.001
IoU threshold	0.70
Classes	5
Cross classes	`false`
NMS results order	`BY_CLASS`
Max bboxes per class	100
Image size	960 × 960

Including preprocessing, inference, and postprocessing on real images, the full YOLOv8n pipeline reaches 60 FPS, with a total runtime of approximately 16.8 ms per frame.

7 Discussion

7.1 Discussion Points

7.1.1 Real-time requirements and conveyor setup

A central question of this project was whether real-time detection is achievable on a low-power platform for a conveyor belt moving at approximately 2 cm/s. From the runtime measurements it is clear that a Raspberry Pi 5 CPU on its own is not sufficient: all nano models operate around ≈ 1 FPS, while the small variants and RT-DETR-R18 fall below that. This is consistent with the theoretical FLOP ratio between the host CPU and the Pi (about $9\times$ difference in FP32 peak throughput) and reflects the limited vectorization and cache capacity on the Raspberry Pi compared to a desktop CPU.

When offloading inference to Hailo-8, the picture changes completely. The compiled YOLOv8n model reaches ≈ 142 FPS in HW-only mode and ≈ 60 FPS for the full pipeline including preprocessing and postprocessing. Even the worst-case accelerator result (YOLOv11s at ≈ 30 FPS) comfortably exceeds the camera's maximum frame rate of ≈ 7 FPS. At 30 FPS, the belt moves less than a millimetre during one inference window, which is negligible compared to the spatial extent of screws and nuts. Thus, from a system level perspective, the use of an external accelerator is not just beneficial but required to satisfy strict real-time constraints on this hardware class.

7.1.2 Model choice and accuracy–speed trade-offs

On GPU and host CPU, the three nano YOLO variants (v8n, v10n, v11n) show very similar runtimes, and the dominant differentiator becomes detection quality. YOLOv8n achieves the highest mAP@0.50 and competitive mAP@0.50–0.95, while YOLOv10n and YOLOv11n trade small variations in mAP against improved parameter efficiency. On the screw dataset, the absolute differences in mAP@0.50–0.95 are modest (on the order of 0.01–0.02), so the choice between these models is more about deployment and tooling convenience than raw accuracy.

On the Raspberry Pi 5, all nano variants converge to roughly the same latency (≈ 1

second per frame), since the CPU is already saturated. RT-DETR-R18 provides a useful transformer-based reference, but its inference time is more than twice that of nano YOLO, and its advantages (e.g. NMS-free decoding, better scaling to large inputs) are not fully exploitable on this constrained platform. In practice, the accuracy gains of heavier models do not justify the additional latency and memory footprint for this use case.

The experiments with input resolution confirm this view. Increasing YOLO11n from 640×640 to 960×960 slightly improves precision and recall for most classes, but the gains in mAP@0.50–0.95 are small. At the same time, the computational cost increases significantly. For the screw domain, where objects occupy a reasonable fraction of the field of view and are well separated, a moderate resolution suffices, and higher resolutions mostly improve robustness for the most difficult class (miscellaneous) rather than fundamentally changing system behaviour.

7.1.3 Class-wise performance and dataset effects

Across all nano models, the best performance is obtained for nut, washer and wood_screw. These classes are visually consistent, are present in sufficient quantity, and exhibit clear shape, which aligns well with the receptive fields and bias of convolutional detectors. The miscellaneous class remains the most challenging: it is both under-represented and deliberately ill-defined, grouping truncated or ambiguous views of screws and nuts into a residual category. This leads to lower confidence scores and more confusion with regular fastener classes, as seen in the confusion matrices.

This behaviour highlights a general limitation of small, domain-specific datasets. Even with augmentation, 2131 images cannot match the diversity of large benchmarks such as COCO, and rare or weakly defined classes are particularly affected. In practice, this suggests that miscellaneous should be interpreted more as a “warning” signal (object present but unreliable classification) than as a precise semantic label.

7.1.4 Quantization and accelerator deployment

The Hailo toolchain performs INT8 post-training quantization (PTQ) based on a calibration set that mirrors the deployment preprocessing. This substantially reduces model size and enables efficient mapping onto Hailo-8, but introduces some degradation in accuracy, especially on small or targets such as screws. In qualitative analysis, the quantized models miss very small or low-contrast objects that the FP32 PyTorch checkpoints detect, or slightly shift box boundaries. This is consistent with the known sensitivity of small

objects to aggressive quantization.

On the other hand, the accelerator-side NMS integration and single- or multi-context mapping greatly reduce host workload and data transfer. For YOLOv8n, the entire network fits into a single context, simplifying scheduling and keeping compilation time low. For YOLOv10 and YOLOv11, the compiler must partition the network into three contexts to meet resource constraints. This increases compilation time and reduces the maximum FPS, but still delivers throughputs far beyond what is required for the conveyor setup.

Overall, the accuracy loss introduced by INT8 PTQ is acceptable for this application, given the substantial throughput gains and the possibility of adjusting detection thresholds or performing occasional human-in-the-loop checks for borderline cases.

7.1.5 Unexplored hardware configurations and deployment possibilities

This project deliberately focuses on a single edge platform (Raspberry Pi 5 with a Hailo-8 accelerator) and a specific deployment pipeline. Several hardware and system-level possibilities remain unexplored but are relevant for industrial adoption:

- **Alternative edge accelerators.** Competing platforms such as NVIDIA Jetson boards, Google Coral (Edge TPU) or other NPU families may offer different trade-offs between peak throughput, power consumption, cost and ease of integration. A systematic comparison under the same dataset and conveyor conditions could change the relative ranking of model architectures.
- **CPU-only optimization on Raspberry Pi.** In this work, the Raspberry Pi 5 CPU is used with a relatively straightforward PyTorch/ONNX runtime. More aggressive optimisation (e.g. graph compilers, operator fusion, pruning, or mixed-precision / INT8 runtimes) might reduce the need of accelerator execution, which would be attractive for extremely cost-sensitive deployments without dedicated NPUs.
- **Microcontroller-class edge devices.** Explore deployment of highly compressed detection models on small microcontrollers (e.g. ARM Cortex-M / RP2040) without an external accelerator, trading frame rate and model complexity for ultra-low power and very low hardware cost.
- **Scaling to higher belt speeds or multiple lines.** All measurements are performed at a belt speed of 2 cm/s and with a single conveyor lane. Higher speeds could re-

quire different batching strategies, more powerful accelerators and different camera configurations.

These unexplored options show that the chosen hardware configuration is one representative point in a much larger design space of edge-deployed systems.

7.1.6 Limitations and threats to validity

Several limitations of this work affect the generality of the conclusions:

- **Narrow domain and controlled environment.** All experiments are conducted on a single conveyor belt, with a fixed camera pose and lighting conditions typical for the lab setup. The results cannot be directly extrapolated to other industrial scenarios with different backgrounds, motion patterns, or illumination.
- **Grayscale, single-camera setting.** The dataset is grayscale and captured from a single top-down viewpoint. While this matches many industrial inspection setups, it does not explore diverse colour configurations that could help overlapping fasteners.
- **Limited dataset size and class diversity.** With 2131 images and five classes the dataset is small by modern detection standards. The reported metrics may be overly optimistic for this specific configuration and may not capture rare failure modes that would appear in large-scale deployment.
- **Focus on a single accelerator family.** The deployment pipeline targets Hailo-8 only. Other NPUs or edge accelerators might have different operator sets, quantization characteristics, or scheduling strategies; replicating the study on alternative hardware could lead to different conclusions about the "best" model architecture.
- **Simplified system-level integration.** The project concentrates on the perception stage (detection) and does not include a full closed-loop integration with a robotic arm. Real-world constraints such as end-to-end latency, mechanical tolerances, and error recovery are therefore only discussed qualitatively.

These limitations motivate several directions for future work, particularly around richer datasets, alternative hardware targets, and integration into a complete sorting system.

8 Conclusion

This specialization project investigated the deployment of lightweight object detectors for real-time fastener detection on constrained edge hardware, specifically Raspberry Pi 5 with and without a Hailo-8 AI accelerator. The target application is a conveyor-belt setup where screws, nuts, washers and ambiguous parts must be detected reliably at a belt speed of approximately 2 cm/s using an industrial IDS uEye camera.

Starting from a literature study on object detection, datasets, model families and accelerator toolchains, a domain-specific grayscale dataset of 2131 annotated images was collected and prepared. Axis-aligned bounding boxes were chosen as the annotation format to align with widely available COCO-pretrained weights and to simplify the export to ONNX and Hailo's HAR/HEF formats. Seven detection models were considered: YOLOv8n/s, YOLOv10n/s, YOLOv11n/s and RT-DETR-R18. All models were fine-tuned via transfer learning and evaluated using standard metrics (IoU, AP, mAP@0.50, mAP@0.50–0.95) and runtime (latency/FPS).

On the host GPU, all YOLO variants achieved strong accuracy on the screw dataset, with mAP@0.50–0.95 around 0.73–0.74 for the nano models and slightly higher values for some small variants. RT-DETR-R18 provided comparable or better accuracy but at substantially higher computational cost. On CPU, the nano YOLO models delivered around 30 FPS on the host machine at 640×640 input, while small models operated near 15 FPS and RT-DETR-R18 around 6–7 FPS. Scaling these results to Raspberry Pi 5 indicated that pure CPU inference would be limited to about 1 FPS (nano), which is insufficient for robust real-time detection in the target scenario.

The deployment on Hailo-8 changed this conclusion: after INT8 post-training quantization and compilation to HEF, YOLOv8n reached over 140 FPS in hardware benchmarks and around 60 FPS for the full pipeline including preprocessing and postprocessing. Other models, such as YOLOv10n and YOLOv11n/s, achieved between 30 and 65 FPS, even when the compiler had to partition them into three contexts. These throughputs are more than adequate for the given conveyor and camera frame rates and effectively remove detection as a bottleneck. Although quantization introduced some accuracy degradation, particularly for the hardest class (miscellaneous) and very small objects, the overall behaviour remained consistent with the FP32 baselines.

Within the constraints of a small, controlled dataset and a single accelerator family, the main conclusions are:

- Nano variants of YOLO (v8/v10/v11) offer an excellent balance between accuracy and computational cost for edge deployments targeting a small number of industrial object classes.
- Raspberry Pi 5 CPU alone cannot sustain robust real-time detection for this use case, but combining it with a dedicated accelerator such as Hailo-8 enables comfortable real-time performance with significant margin.
- INT8 post-training quantization, when guided by a realistic calibration set and integrated NMS, is a practical approach to deploying modern detectors on NPUs, although small objects and ambiguous classes remain sensitive.
- A carefully designed, domain-specific dataset—combined with transfer learning from large-scale COCO-pretrained weights—can yield strong performance even with relatively few images, provided that the deployment conditions closely match the data collection setup.

Overall, the work demonstrates that real-time fastener detection is feasible on embedded systems when hardware acceleration is available, and it provides a concrete reference pipeline (camera $\rightarrow$ dataset $\rightarrow$ training $\rightarrow$ ONNX/HAR/HEF $\rightarrow$ Pi+Hailo) that can be reused for similar Edge-AI projects.

9 Future Work

- **Quantization-aware and mixed-precision training.**
  Explore quantization-aware training (QAT) or mixed-precision schemes (e.g. INT8 activations with higher-precision weights) to reduce the accuracy gap between FP32 and Hailo INT8 deployment, particularly for small and low-contrast objects.

- **Exploration of alternative accelerators and backends.**
  Port the pipeline to other edge accelerators (e.g. Coral Edge TPU, NVIDIA Jetson, or other NPUs) and compare end-to-end performance, toolchain maturity, and power consumption. This would clarify how tightly the conclusions depend on the Hailo-specific compilation flow.

- **End-to-end integration with robotic manipulation.**
  Integrate the detection system with a robotic arm or pick-and-place mechanism and evaluate closed-loop performance: latency from image capture to actuation, success rate of grasps, and robustness to misdetections over long runs.

- **Adaptive scheduling and multi-camera setups.**
  Investigate multi-camera configurations (e.g. side views) and adaptive scheduling strategies where detector resolution or frame rate is adjusted dynamically based on belt load, object density, or confidence estimates.

- **Model compression and architecture search.**
  Apply pruning, knowledge distillation or neural architecture search (NAS) to derive custom detectors tailored specifically to the screw domain and Hailo-8 hardware constraints, potentially outperforming generic YOLO variants in both speed and accuracy.

- **Monitoring, calibration and maintenance in deployment.**
  Design tools for online monitoring of detection quality, automatic threshold adjustment, and re-calibration when the environment drifts (e.g. belt wear, lighting changes, new fastener types), bridging the gap between lab prototype and production system.

10 Description of Use of Artificial Intelligence

In working on this specialization project, I have used artificial intelligence (AI) tools in the following limited ways. I used ChatGPT (OpenAI) to assist with idea development, clarifying concepts, and improving the structure and language of the text (for example, by suggesting alternative formulations, checking grammar, and helping to simplify explanations). AI was also used to get help with technical formatting issues in LaTeX and to debug small code examples by explaining error messages and proposing corrections.

All scientific content, including the problem formulation, choice of methods, implementation, experiments, analysis, and conclusions, has been developed, reviewed, and academically assessed by me. AI tools were not used to generate entire sections of the report, to perform literature searches independently, or to answer the assignment in its entirety. I have not uploaded sensitive or confidential information to the AI tools. I am solely responsible for the academic content of this assignment.

Bibliography

[1] M. AI, "Meta llama 3." https://github.com/meta-llama/llama3, 2024. Accessed: 2025-11-12.

[2] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "Imagenet classification with deep convolutional neural networks," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2012.

[3] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

[4] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. MIT Press, 2016.

[5] T.-Y. Lin, M. Maire, S. Belongie, J. Hays, P. Perona, D. Ramanan, P. Dollár, and C. L. Zitnick, "Microsoft coco: Common objects in context," in *European Conference on Computer Vision (ECCV)*, Springer, 2014.

[6] Ultralytics, "Yolov5 quickstart tutorial." https://docs.ultralytics.com/yolov5/quickstart_tutorial/. Accessed: 2025-10-29.

[7] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You only look once: Unified, real-time object detection," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016.

[8] Flickr user, original photographer (via Microsoft COCO), "Coco 2017 validation image id 285 (bear)." http://farm8.staticflickr.com/7434/9138147604_c6225224b8_z.jpg, 2017. Licensed under Creative Commons Attribution 2.0 (CC BY 2.0), http://creativecommons.org/licenses/by/2.0/. Image ID 285 from COCO 2017 validation set; annotations licensed under CC BY 4.0.

[9] G.-S. Xia, X. Bai, J. Ding, Z. Zhu, S. Belongie, J. Luo, M. Datcu, M. Pelillo, and L. Zhang, "Dota: A large-scale dataset for object detection in aerial images," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018.

[10] Ultralytics, "Yolo11: Model card and benchmarks." https://github.com/ultralytics/ultralytics. Accessed: 2025-10-29.

[11] S. Shao, Z. Li, T. Zhang, C. Peng, G. Yu, X. Zhang, J. Li, and J. Sun, "Objects365: A large-scale, high-quality dataset for object detection," in *Proceedings of the IEEE/CVF international conference on computer vision*, pp. 8430–8439, 2019.

[12] A. Kuznetsova, H. Rom, N. Alldrin, J. Uijlings, I. Krasin, J. Pont-Tuset, S. Kamali, S. Popov, M. Malloci, A. Kolesnikov, *et al.*, "The open images dataset v4: Unified image classification, object detection, and visual relationship detection at scale," *International journal of computer vision*, vol. 128, no. 7, pp. 1956–1981, 2020.

[13] M. Ulrich, P. Follmann, and J.-H. Neudeck, "A comparison of shape-based matching with deep-learning-based object detection," *tm-Technisches Messen*, vol. 86, no. 11, pp. 685–698, 2019.

[14] M. S. GmbH, "Mvtec screws dataset (version 1.0)." `https://www.mvtec.com/research`, 2020. Released with HALCON 19.05. Licensed under Creative Commons Attribution-NonCommercial-ShareAlike 4.0 International (CC BY-NC-SA 4.0), `https://creativecommons.org/licenses/by-nc-sa/4.0/`. Contains 384 images of 13 screw and nut categories with oriented bounding box annotations.

[15] M. A. Sadeghi and D. Forsyth, "30hz object detection with dpm v5," in *Computer Vision – ECCV 2014*, pp. 65–79, Springer, 2014.

[16] J. Yan, Z. Lei, L. Wen, and S. Z. Li, "The fastest deformable part model for object detection," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 2497–2504, IEEE, 2014.

[17] K. Lenc and A. Vedaldi, "R-cnn minus r," *arXiv preprint arXiv:1506.06981*, 2015.

[18] R. B. Girshick, "Fast r-cnn," *arXiv preprint arXiv:1504.08083*, 2015.

[19] S. Ren, K. He, R. Girshick, and J. Sun, "Faster r-cnn: Towards real-time object detection with region proposal networks," *arXiv preprint arXiv:1506.01497*, 2015.

[20] R. Khanam and M. Hussain, "Yolov11: An overview of the key architectural enhancements," *arXiv preprint arXiv:2410.17725*, 2024.

[21] Ultralytics, "Explore ultralytics yolov8," 2023.

[22] C.-Y. Wang *et al.*, "YOLOv10: Real-time end-to-end object detection," *arXiv preprint*, 2024. arXiv:2405.14458.

[23] J. Lv *et al.*, "RT-DETR: Real-time detection transformer," *arXiv preprint*, 2023. arXiv:2304.08069.

[24] RangiLyu, "Nanodet-plus: Super fast and high accuracy lightweight anchor-free object detection model.." https://github.com/RangiLyu/nanodet, 2021.

[25] Hailo Technologies Ltd., *Hailo Dataflow Compiler User Guide*. User guide.

[26] S. Han, "6.5940: Tinyml and efficient deep learning computing." https://www.youtube.com/@efficientml-ai, 2024. EfficientML.ai lecture series.

[27] D.-I. B. Hammoud, "Embedded machine learning: Chapter 12—ai hardware accelerators." Lecture 12, Department of Electrical and Computer Engineering, RPTU Kaiserslautern–Landau, 7 2025.

[28] Roboflow, Inc., "Roboflow: End-to-end computer vision platform." https://roboflow.com, 2019. Accessed: 2025-11-03.

[29] EmbeddedAIProject, "Embeddedaiproject dataset." https://universe.roboflow.com/embeddedaiproject/embeddedaiproject-id4jh, oct 2025. visited on 2025-11-04.

[30] IDS Imaging Development Systems GmbH, *UI-3590CP-C-HQ Rev.2 (AB00606) uEye Industrial Camera Datasheet*, 2025. Datasheet, accessed: 2025-11-04.

[31] IDS Imaging Development Systems GmbH, "pyueye: Python bindings for ueye api." https://pypi.org/project/pyueye/, 2022. Version 4.96.952.

[32] G. Jocher, A. Chaurasia, and J. Qiu, "Ultralytics yolov8," 2023.

A Demonstration (Appendix A)

Figure A.1: All types of Fasteners used for detection

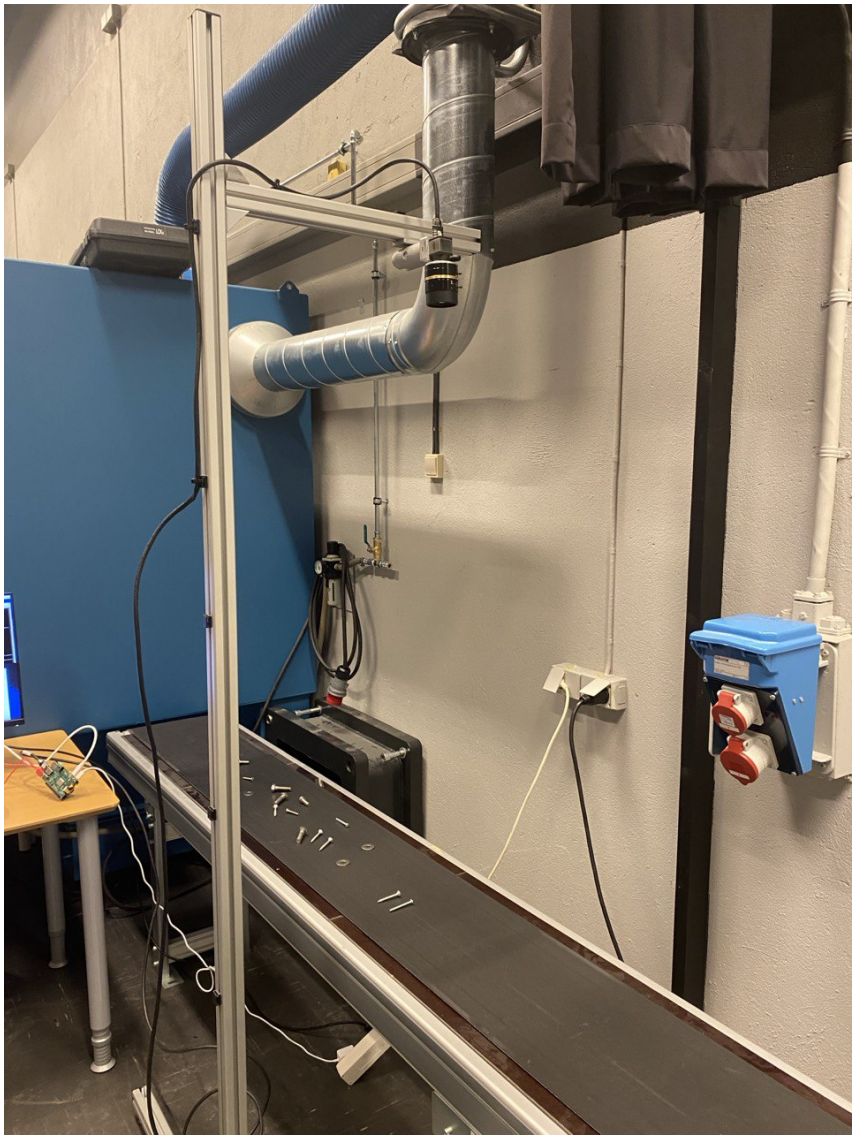


Figure A.2: The conveyor belt

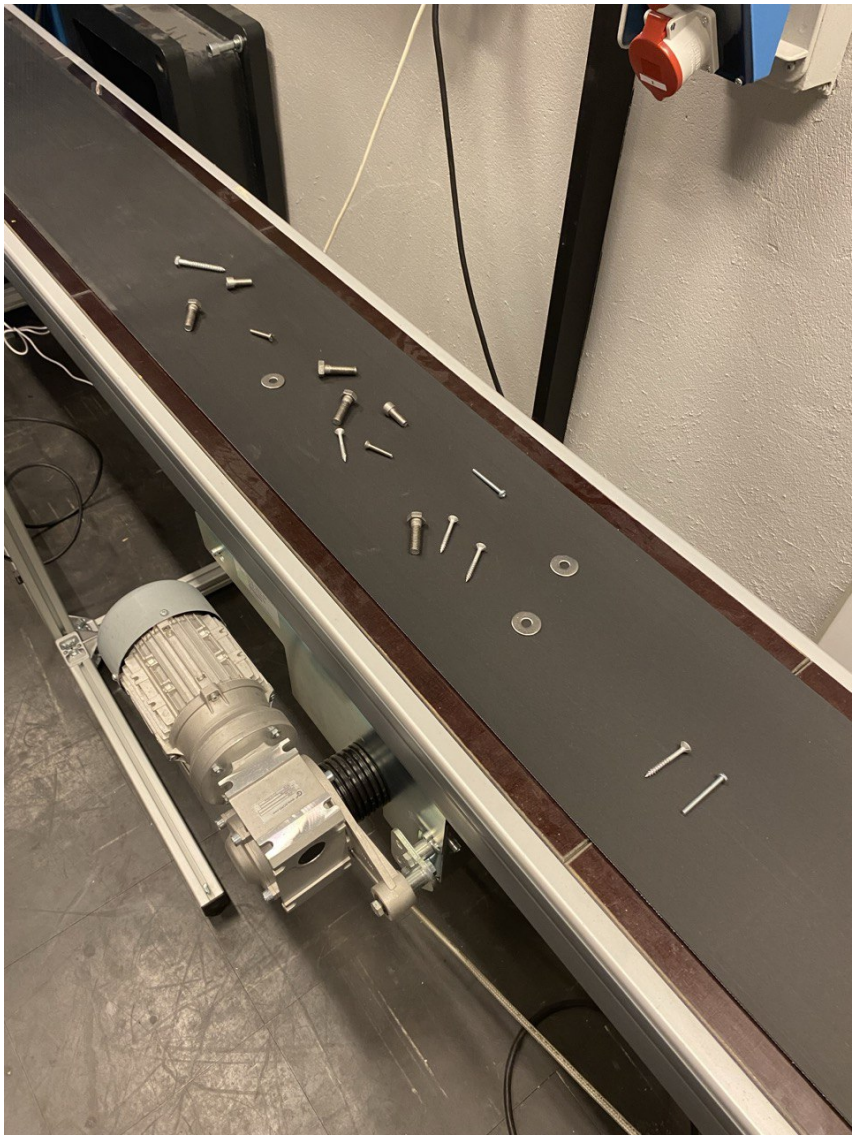


Figure A.3: The conveyor belt with fasteners

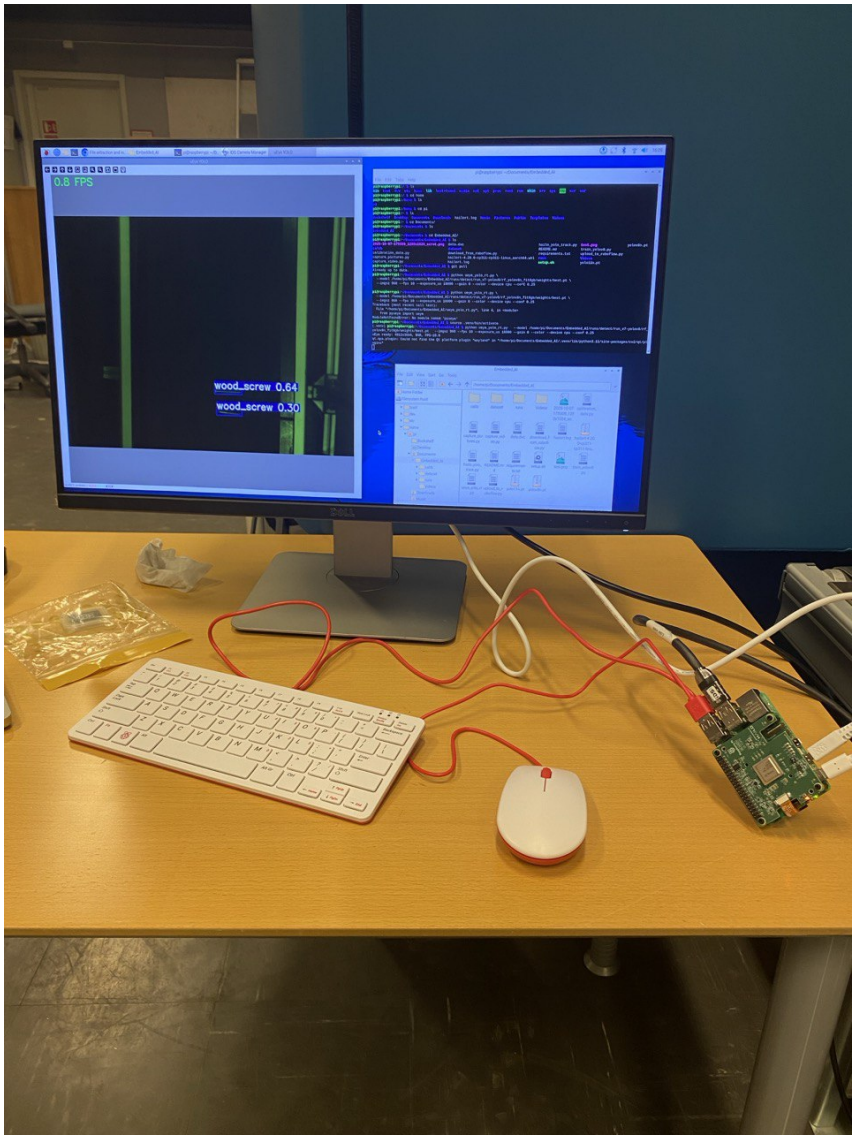


Figure A.4: The Raspberry Pi 5 with Performed Streeming

B Hailo Compilation SetUp (Appendix B)

Listing B.1: Export YOLO to ONNX for Hailo toolchain

```
# ONNX export settings, friendly for accelerator toolchains
yolo export \
  model=.../best.pt \
  format=onnx \
  imgsz=960,960 \
  opset=12 \
  dynamic=False \
  simplify=True \
  nms=False
```

Listing B.2: Parse ONNX to HAR with Hailo Translation API

```
hailo parser onnx \
  --hw-arch hailo8 \
  --net-name yolov11n_custom \
  --har-path .../yolov11n_parsed.har \
  --input-format NCHW \
  --tensor-shapes '[1,3,960,960]' \
  -y \
  .../best.onnx
```

Hailo compiler parsed ONNX file with configuration:

- NMS added on the Hailo side
- YOLO models output three or six detection heads (depending on stride levels). But many exported ONNX graphs also include a concatenation node that merges these heads into one big tensor. Hailo wants the raw outputs before that concat, because this allows:
  - better scheduling on hardware
  - Hailo's own optimized NMS
  - less memory overhead
  - cleaner graph structure

- Hailo recommends using Conv head outputs as end nodes:
  - /model.23/cv2.0/cv2.0.2/Conv
  - /model.23/cv3.0/cv3.0.2/Conv
  - /model.23/cv2.1/cv2.1.2/Conv
  - /model.23/cv3.1/cv3.1.2/Conv
  - /model.23/cv2.2/cv2.2.2/Conv
  - /model.23/cv3.2/cv3.2.2/Conv

Listing B.3: Hailo optimization with calibration set

```
hailo optimize --hw-arch hailo8 \
  --calib-set-path .../calib_nhwc_960.npy \
  --output-har-path .../yolo11n_quant.har \
  .../yolov11n_parsed.har
```

For YOLO models, the Hailo compiler (DFC 3.33.0) recognized a YOLOv8-like NMS structure but did not allow running the post-processing on the NN-core for this meta-architecture. Therefore, we offload NMS to the Raspberry Pi CPU, while the backbone and detection heads run on Hailo-8.

Listing B.4: Compile quantized YOLO11n HAR to HEF on Hailo-8

```
hailo compiler \
  --hw-arch hailo8 \
  --output-dir ... \
  .../yolo11n_quant.har
```